

Hive

Agustín C. Caminero Herráez
Depto. de Sistemas de Comunicación y Control.
UNED
accaminero@scc.uned.es / @accaminero

ÍNDICE

- ¿Por qué hay herramientas de programación de alto nivel?
- Hive vs. Pig
- Introducción a Hive.
- Arquitectura de Hive.
- Ejemplo de funcionamiento.
- Trabajando con una base de datos.
- Modelo de datos.
- Trabajando con tablas.
- Lenguaje de manipulación de datos.
- Reuniones.
- Formatos de serialización y deserialización (serde).
- Planes de ejecución de consultas.
- Transacciones.
- Otros aspectos interesantes de Hive.

OBJETIVOS

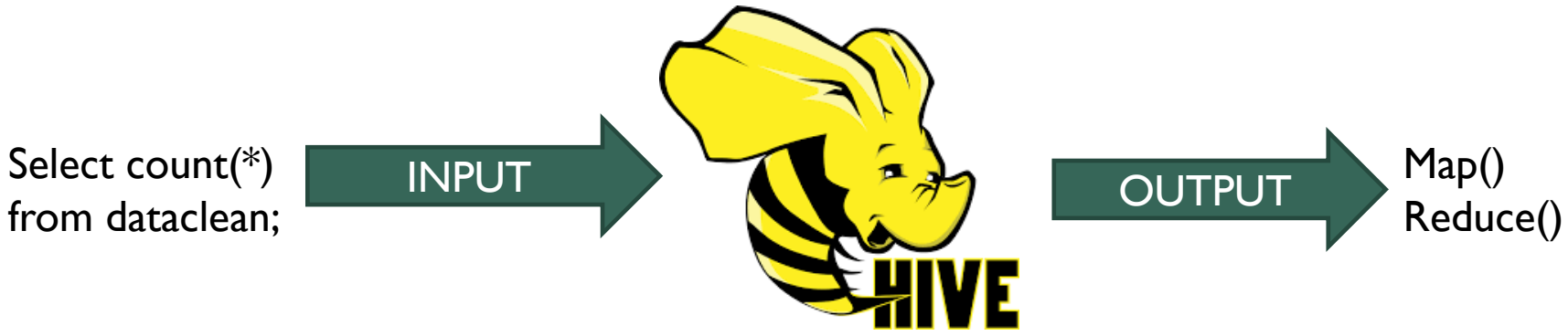
- Comprender qué es Hive.
- Comprender la arquitectura de Hive.
- Comprender las diferencias entre los distintos tipos de tablas.
- Diseñar bases de datos, tablas y consultas utilizando HiveQL.
- Comprender que es posible programar MapReduce utilizando un lenguaje de consultas similar a SQL.

¿POR QUÉ HAY HERRAMIENTAS DE PROGRAMACIÓN DE ALTO NIVEL?

- Programar MapReduce es una tarea tediosa y compleja.
  - Es necesario particionar el problema en funciones map y reduce, definir las claves, …
- Los lenguajes de alto nivel (Hive y Pig) tratan de proporcionar la misma funcionalidad, y aportan las siguientes ventajas:
  - Código más reducido y limpio.
  - Reducen el tiempo de desarrollo.

¿POR QUÉ HAY HERRAMIENTAS DE PROGRAMACIÓN DE ALTO NIVEL?

- Pig y Hive toman como entrada un programa en un lenguaje de alto nivel y producen como salida código MapReduce



¿POR QUÉ HAY HERRAMIENTAS DE PROGRAMACIÓN DE ALTO NIVEL?

- Ejemplo wordcount:

```
create table doc(
text string
) row format delimited fields terminated by '\n' stored as textfile;

load data inpath '/tmp/texto' overwrite into table doc;

SELECT word, COUNT(*) FROM doc LATERAL VIEW explode(split(text, '  ')) tmpTable as word GROUP BY word;
```

HIVE VS. PIG

- El ecosistema de Hadoop proporciona una variedad de herramientas de programación, dos de las más populares son Hive y Pig.
- Estas herramientas permiten programar aplicaciones MapReduce sin tener que diseñar de forma explícita las funciones map y reduce.
  - MapReduce tiene gran complejidad y una curva de aprendizaje alta, **que se evitan.**





HIVE VS. PIG

- Aunque aparentemente ambas herramientas sirven para lo mismo, pero con diferencias.

Pig	Hive
Lenguaje procedural.	Lenguaje declarativo similar a SQL.
Datos des/estructurados	Centrado en datos estructurados.
Esquema definido en el programa.	Esquema definido de antemano.
Principalmente usado para programar.	Principalmente usado para generar informes.

HIVE

- Hive es una infraestructura de data warehouse sobre Hadoop que permite la utilización de un cluster de Hadoop sin necesidad de programar MapReduce.
- Utiliza un lenguaje de consultas similar a SQL, llamado HiveQL, de forma que desarrolladores que conozcan SQL pueden utilizar Hadoop.
- Hive **«compila»** las consultas redactadas en HiveQL para crear las tareas MapReduce.

Select count(*)
from dataclean;



Map()
Reduce()

HIVE

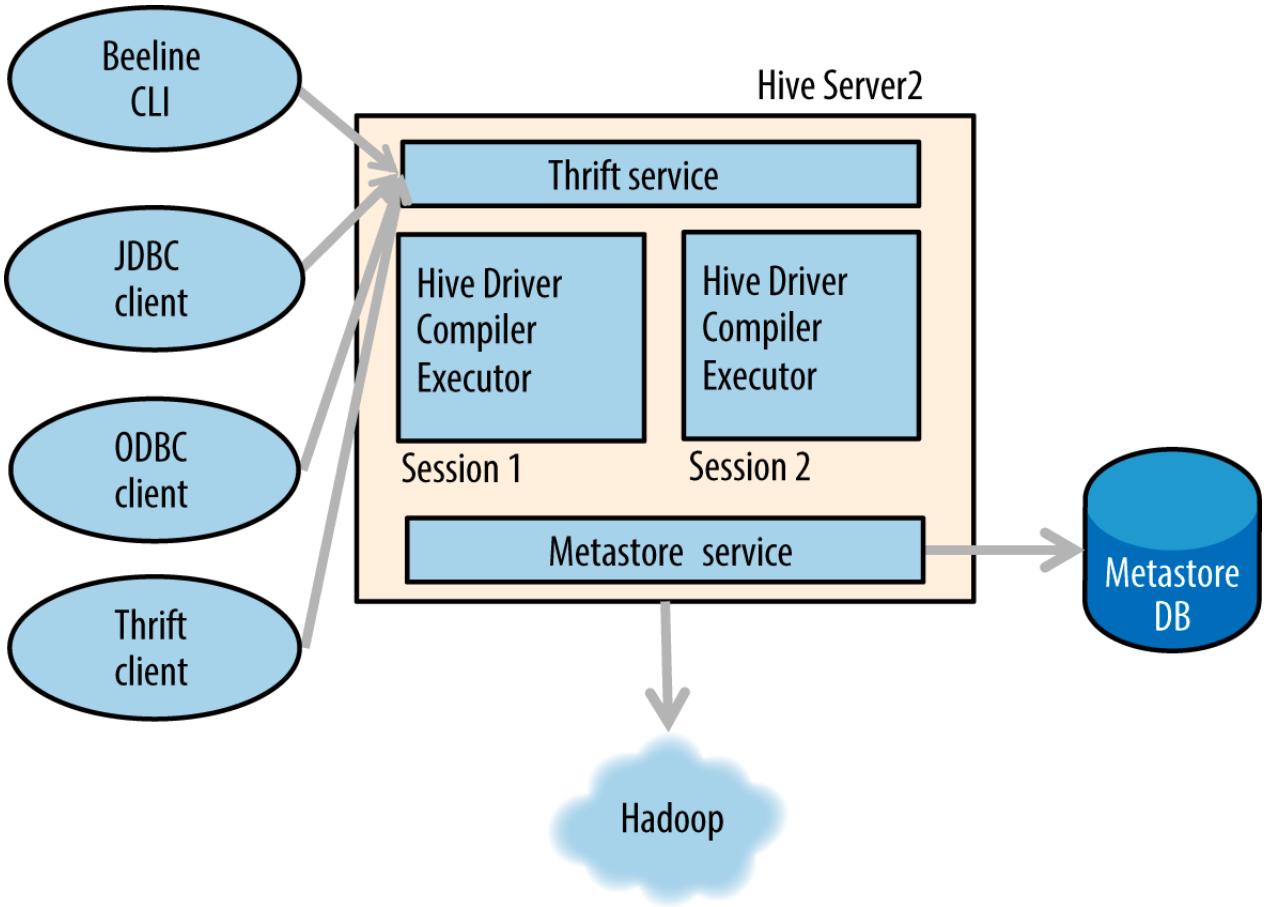
- Entre sus ventajas más importantes se encuentran:
  - Es extensible: Permite entre otros utilizar tipos de datos y funciones definidos por el usuario, utilizar plugins desarrollados en otros lenguajes.
  - Es interoperable: Soporta diferentes formatos de datos y de ficheros.
  - Ofrece altas prestaciones y tolerancia a fallos: Aprovecha las características de Hadoop.

HIVE

- ¿Qué diferencias existen entre Hadoop con Hive y los sistemas gestores de bases de datos relacionales (RDBMS) tradicionales?

Hadoop con Hive	RDBMS
La escalabilidad es su fuerte.	Escalan peor.
Diseñado para gestionar consultas de alta latencia.	Diseñados para gestionar consultas SQL de alta y baja latencia.
Trabajan sobre petabytes de datos multiestructurados.	Trabajan sobre varios terabytes de datos estructurados.
Eficiente en cuanto a costes.	Alto coste de escalado.

ARQUITECTURA DE HIVE

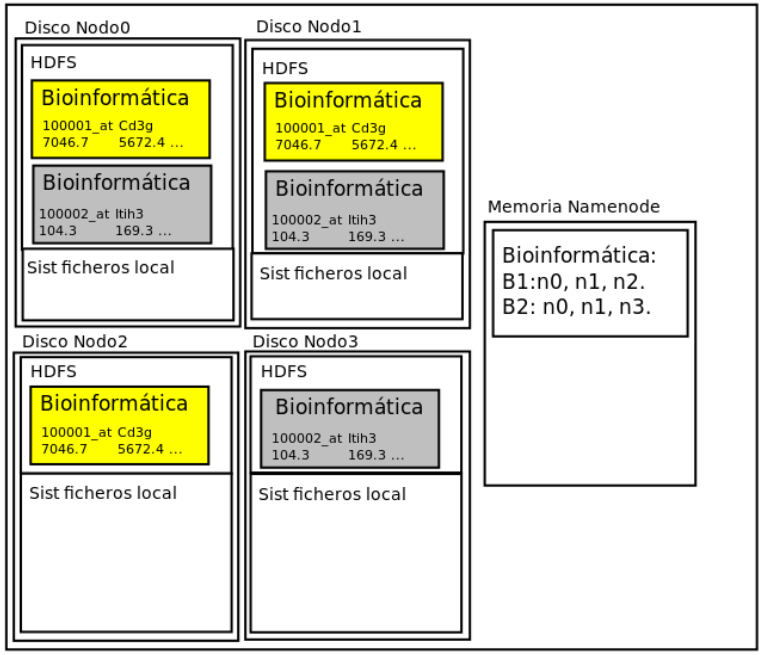


ARQUITECTURA DE HIVE

- Hiveserver2:
  - Permite conexiones remotas utilizando ODBC/JDBC, de forma que se pueden utilizar aplicaciones tales como Excel, Toad o SQuirreL.
  - Estas conexiones las gestiona el componente Thrift.
  - Metastore:
    - Contiene las definiciones de los esquemas de base de datos.
    - Es una base de datos relacional, externa a Hive.
- Clientes:
  - Terminal: beeline.
  - Interfaz gráfico web: HUE.
  - Herramientas legadas: Toad, SQuirreL… a través de JDBC/ODBC.
- Además, otro componente es HCatalog:
  - Permite que otras herramientas (ej. Pig) puedan leer y escribir esquemas de datos almacenados en el metastore.

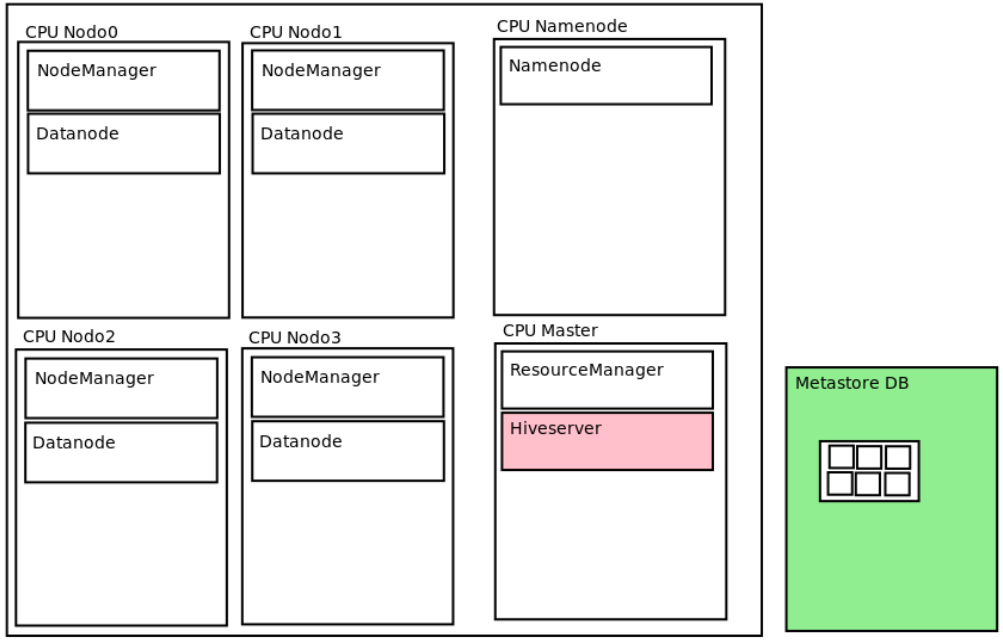
CLUSTER

Almacenamiento (HDFS)

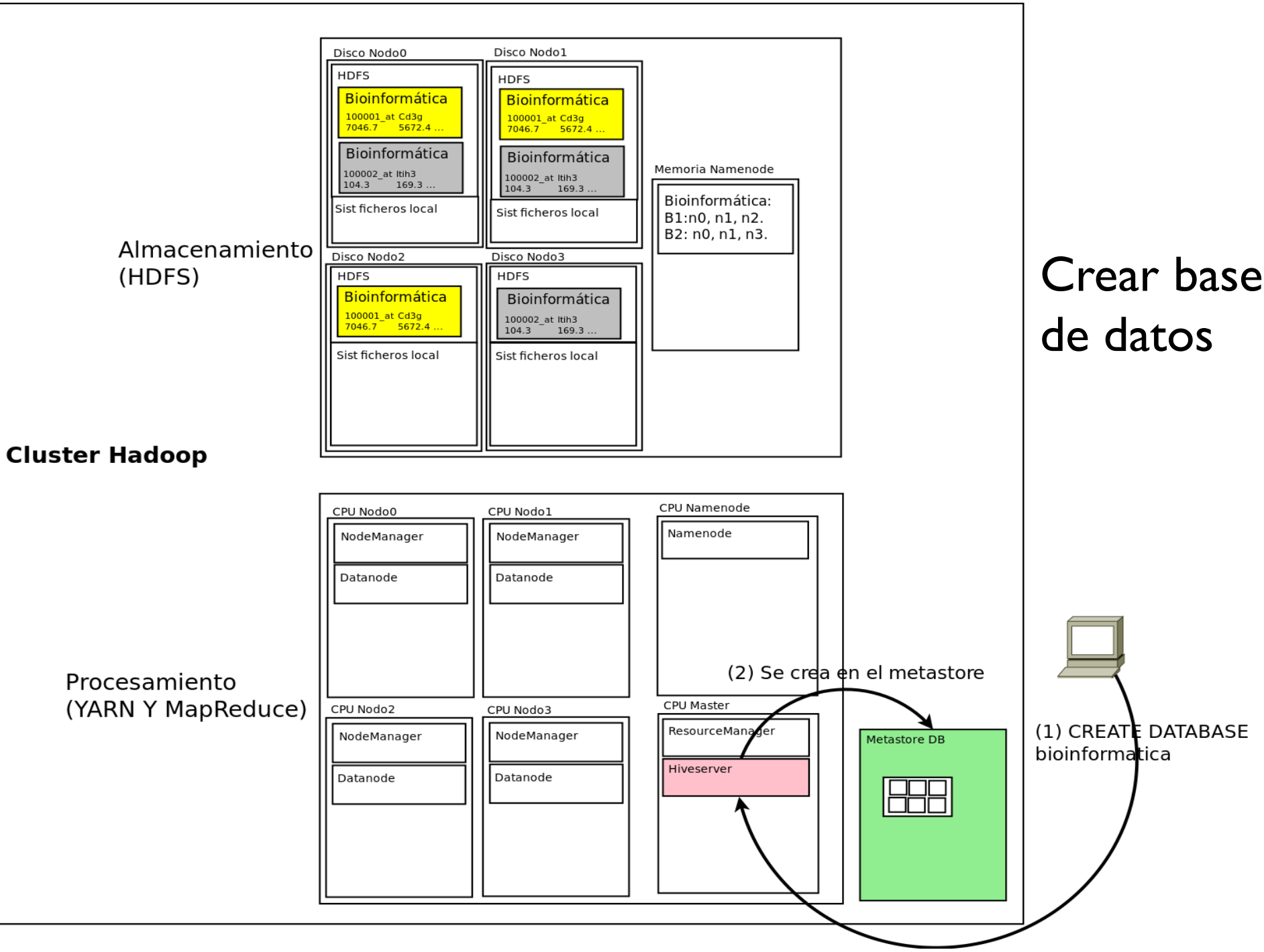


Cluster Hadoop

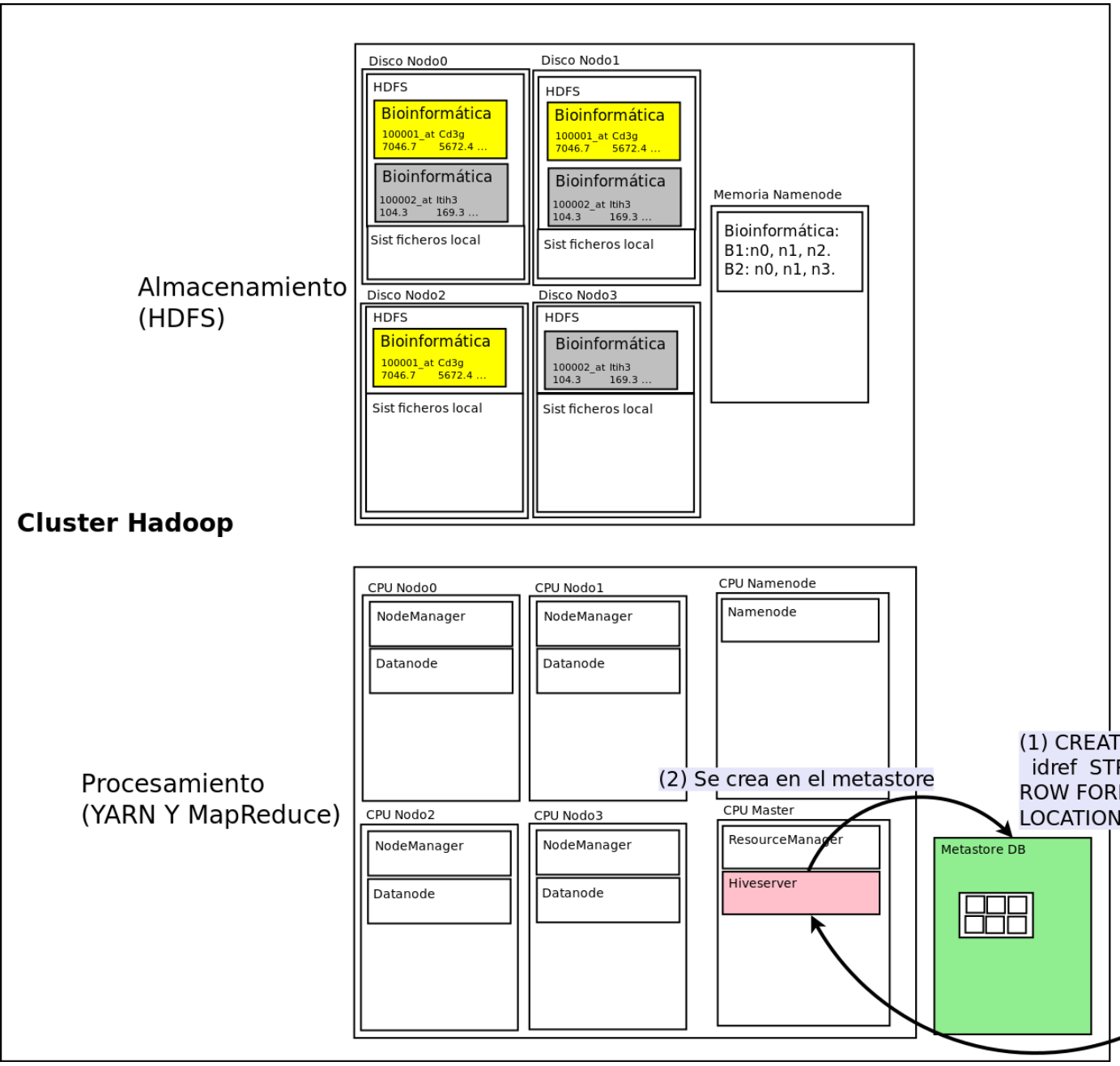
Procesamiento (YARN Y MapReduce)



Crear base de datos

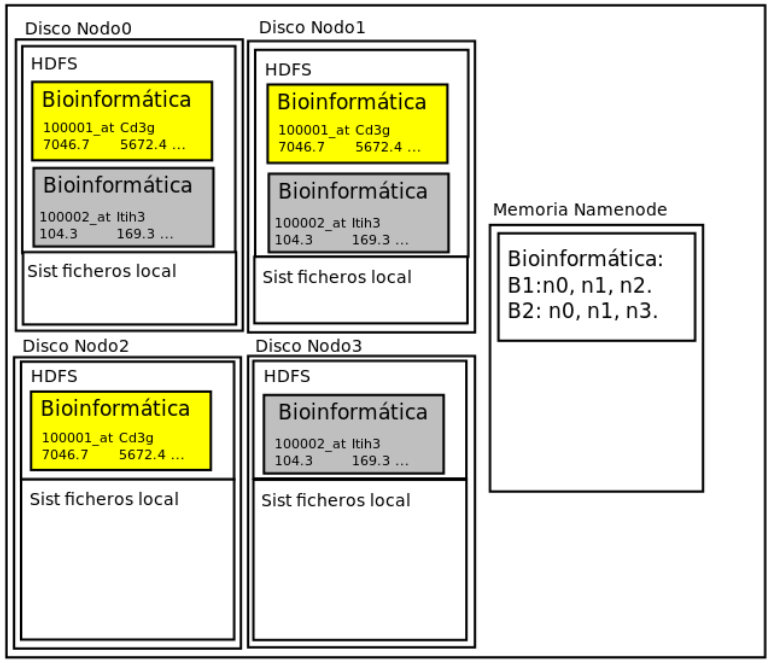


Crear tabla


Almacenamiento
(HDFS)
Disco Nodo0
HDFS
Bioinformática
100001_at Cd3g
7046.7 5672.4 ...
Bioinformática
100002_at Itih3
104.3 169.3 ...
Sist ficheros local
Disco Nodo1
HDFS
Bioinformática
100001_at Cd3g
7046.7 5672.4 ...
Bioinformática
100002_at Itih3
104.3 169.3 ...
Sist ficheros local
Memoria Namenode
Bioinformática:
B1:n0, n1, n2.
B2: n0, n1, n3.
Disco Nodo2
HDFS
Bioinformática
100001_at Cd3g
7046.7 5672.4 ...
Sist ficheros local
Disco Nodo3
HDFS
Bioinformática
100002_at Itih3
104.3 169.3 ...
Sist ficheros local
Cluster Hadoop
Procesamiento
(YARN Y MapReduce)
CPU Nodo0
NodeManager
Datanode
CPU Nodo1
NodeManager
Datanode
CPU Namenode
Namenode
CPU Nodo2
NodeManager
Datanode
CPU Nodo3
NodeManager
Datanode
CPU Master
ResourceManager
Hiveserver
(2) Se crea en el metastore
Metastore DB
(1) CREATE EXTERNAL TABLE datacleanexternal (
idref STRING,ident STRING, gsm19023 FLOAT...)
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t
LOCATION '/ruta/al/fichero/bioinformatica';

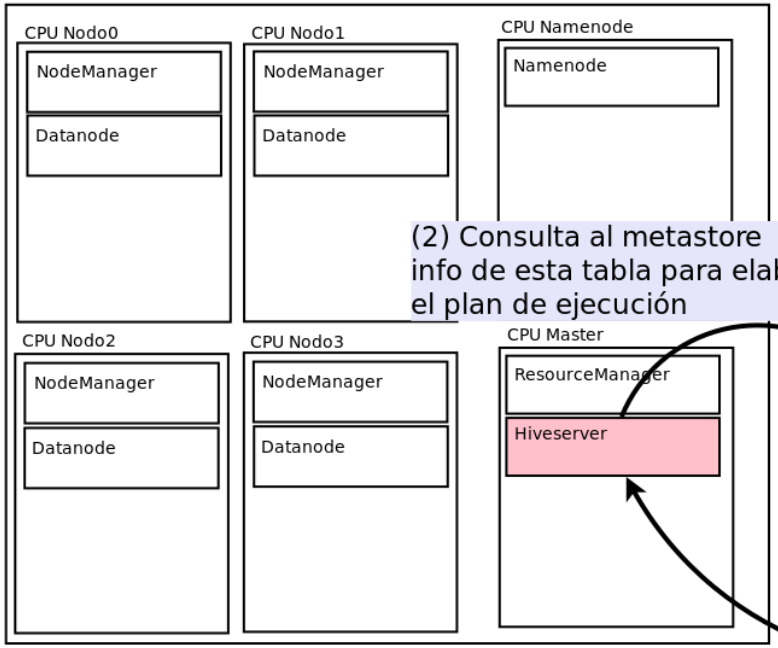
Select

Almacenamiento
(HDFS)



Cluster Hadoop

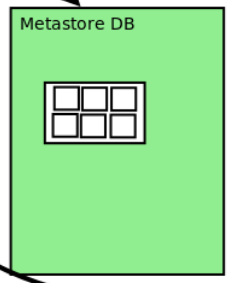
Procesamiento
(YARN Y MapReduce)



(2) Consulta al metastore
info de esta tabla para elaborar
el plan de ejecución

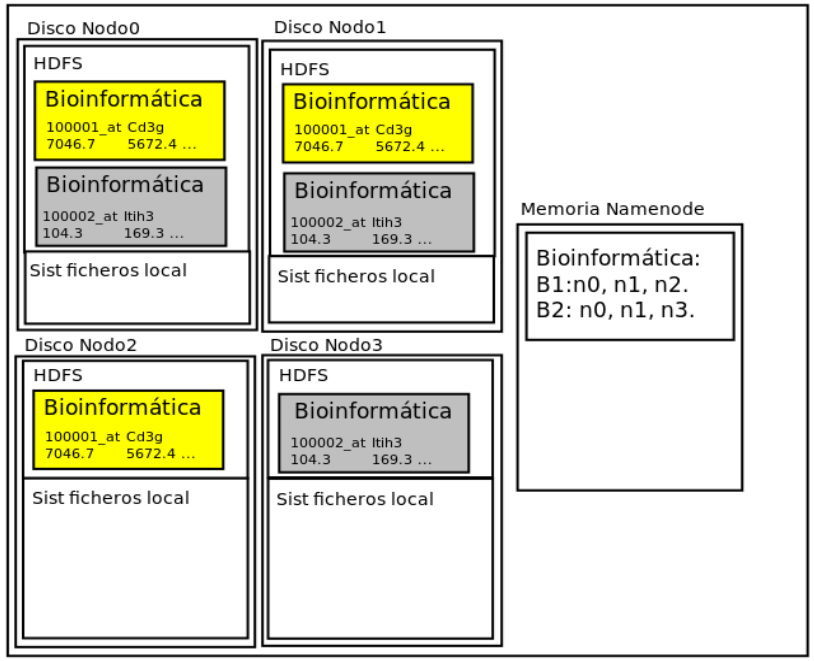


(1) SELECT COUNT(gsd19024)
FROM datacleaneternal
WHERE genetitle='angiopoietin-like 4'



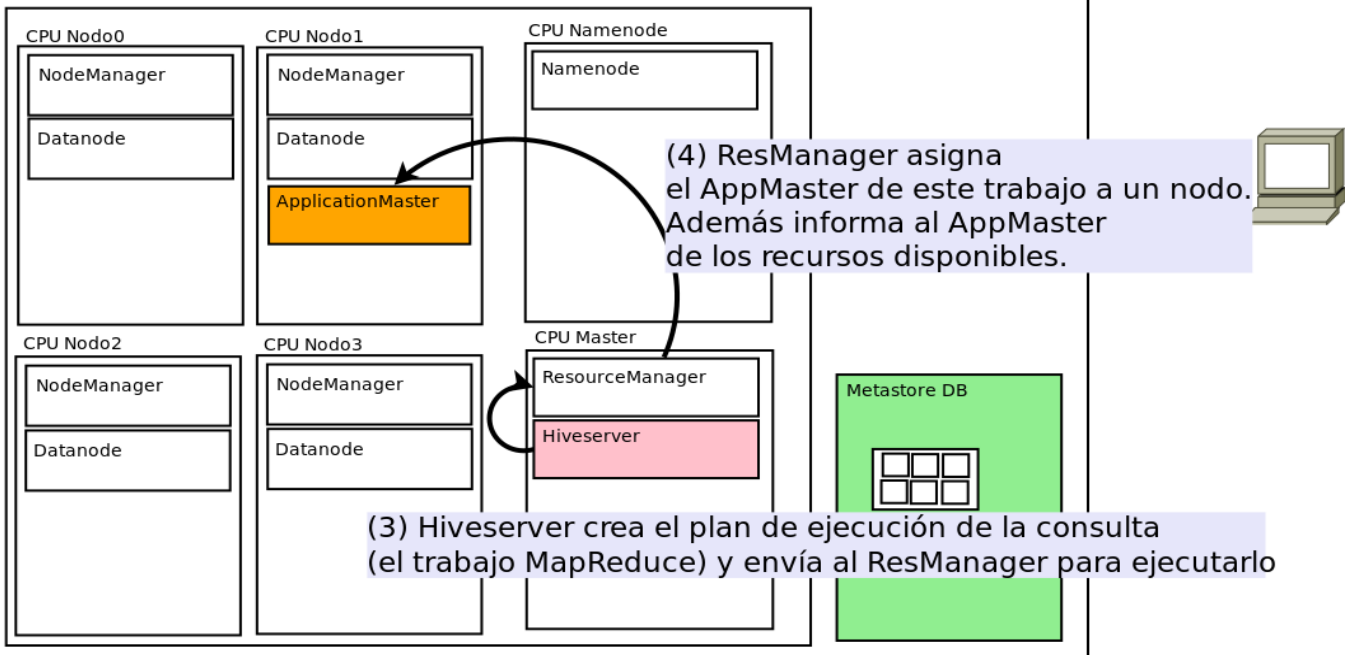
Select

Almacenamiento
(HDFS)



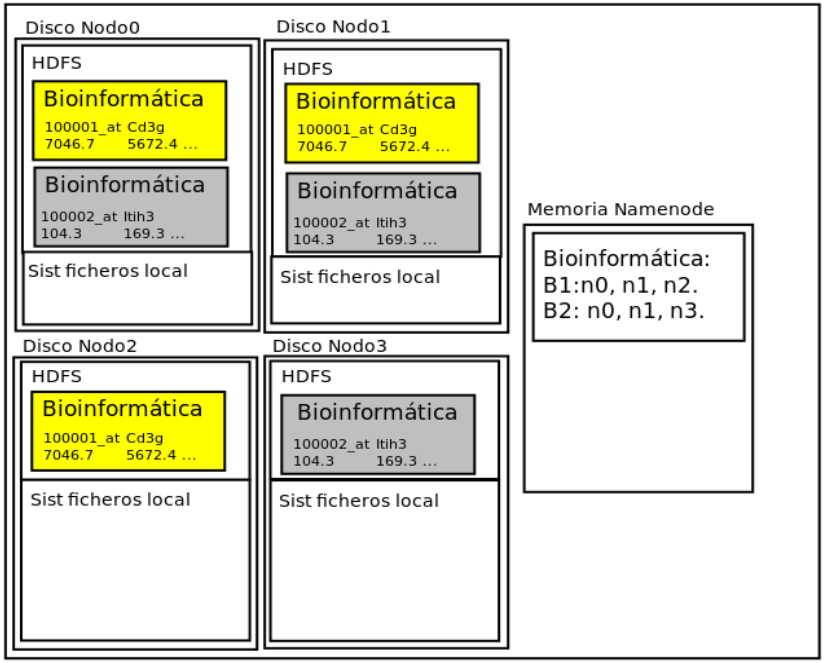
Cluster Hadoop

Procesamiento
(YARN Y MapReduce)



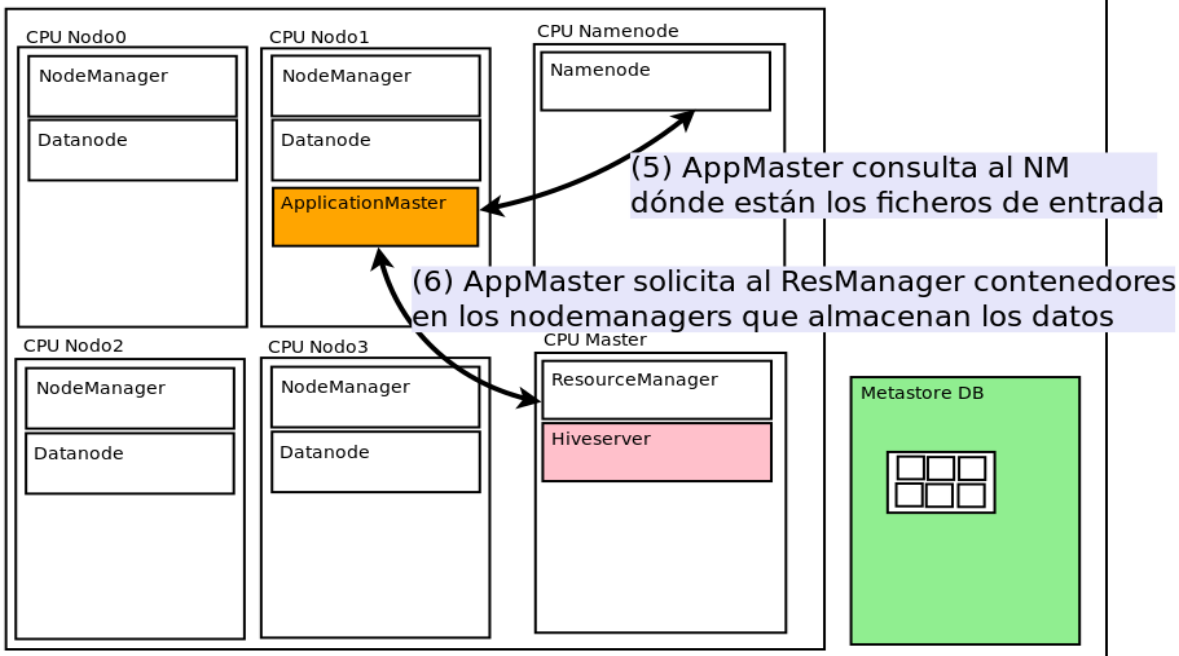
Select

Almacenamiento
(HDFS)



Cluster Hadoop

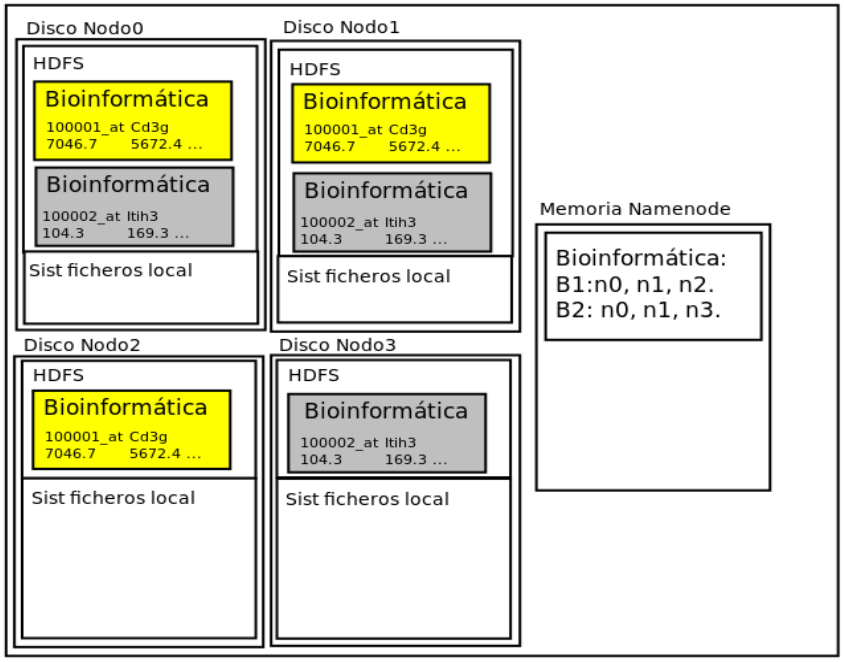
Procesamiento
(YARN Y MapReduce)





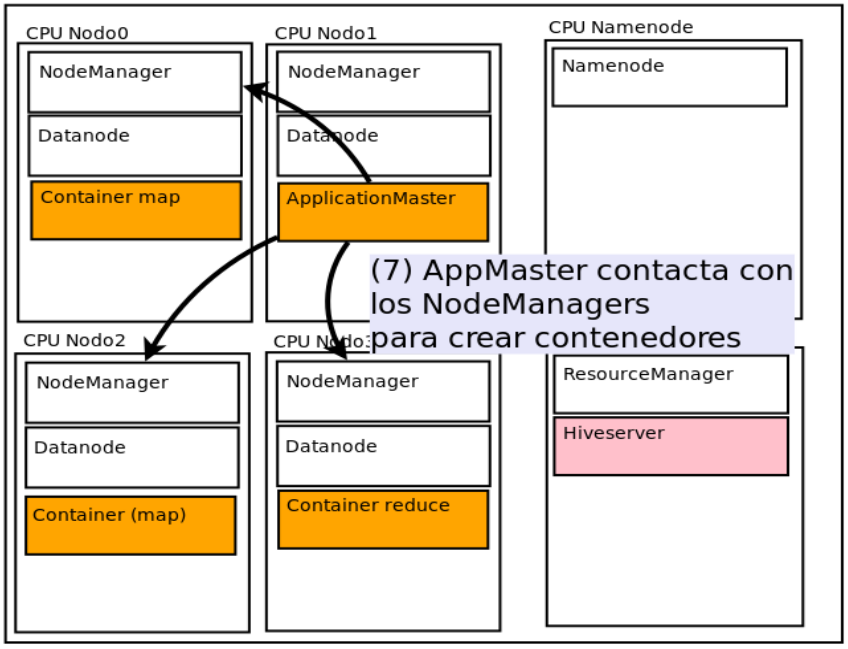
Select

Almacenamiento
(HDFS)



Cluster Hadoop

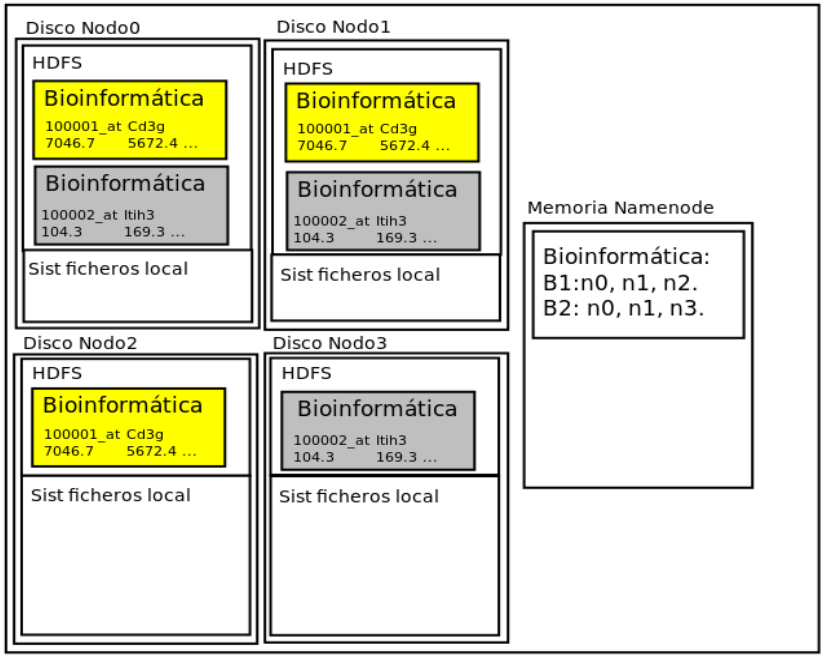
Procesamiento
(YARN Y MapReduce)





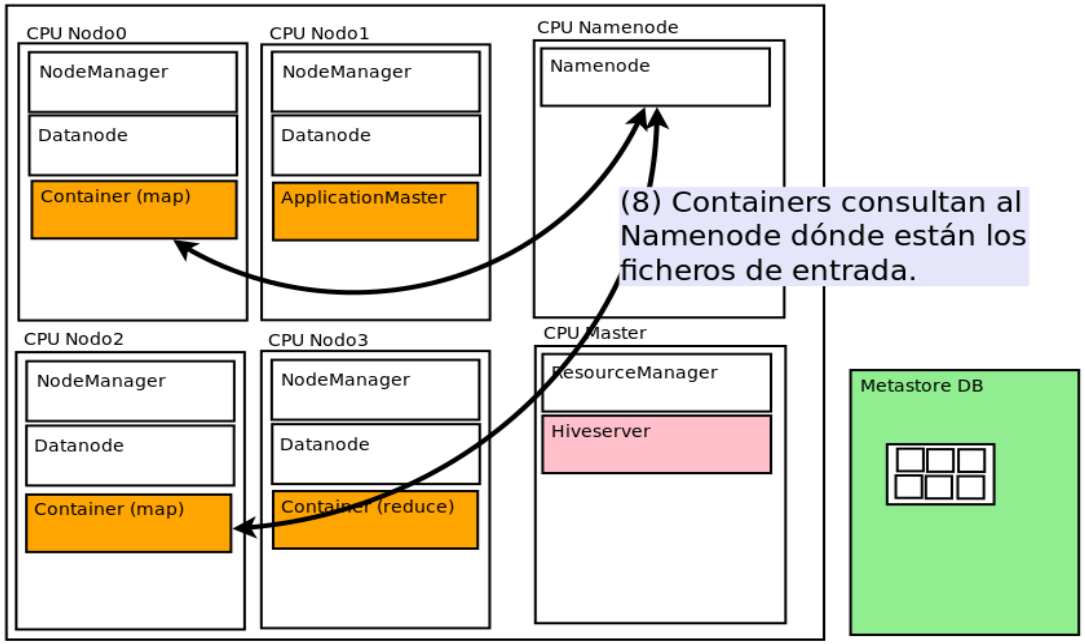
Select

Almacenamiento
(HDFS)



Cluster Hadoop

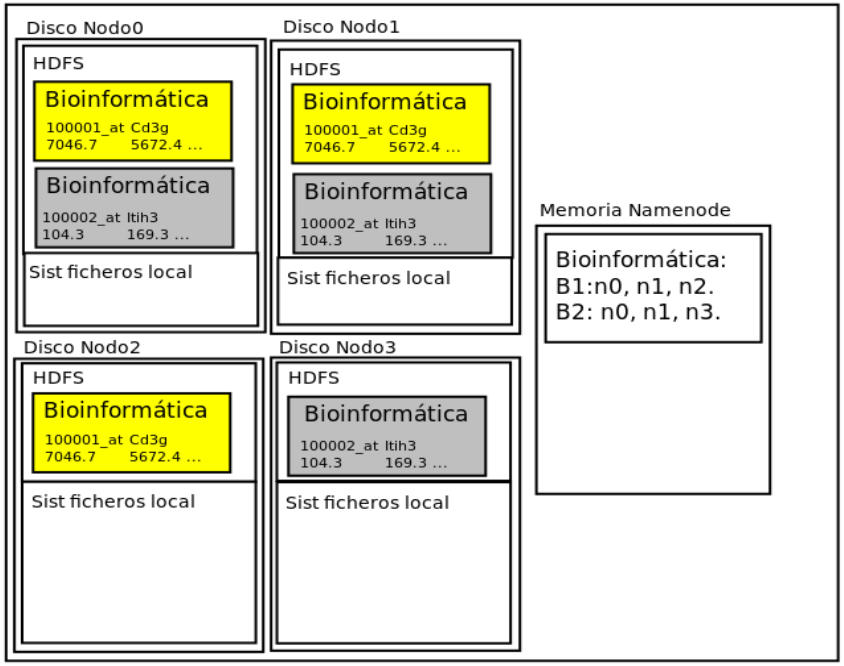
Procesamiento
(YARN Y MapReduce)





Select

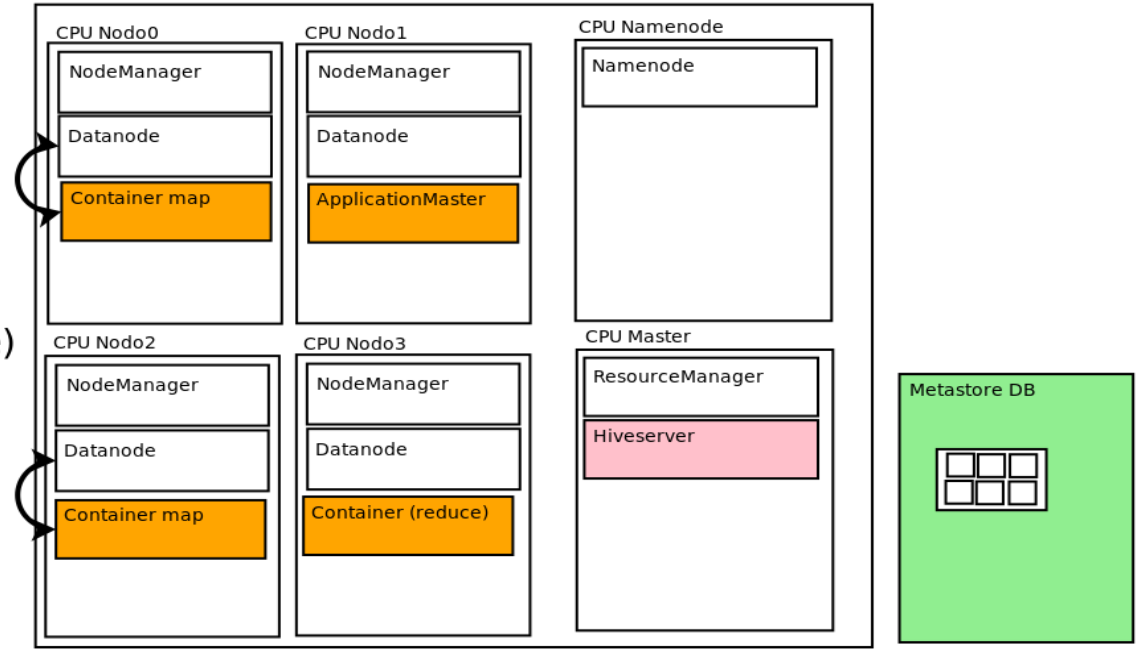
Almacenamiento
(HDFS)

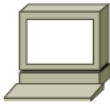


Cluster Hadoop

Procesamiento
(YARN Y MapReduce)

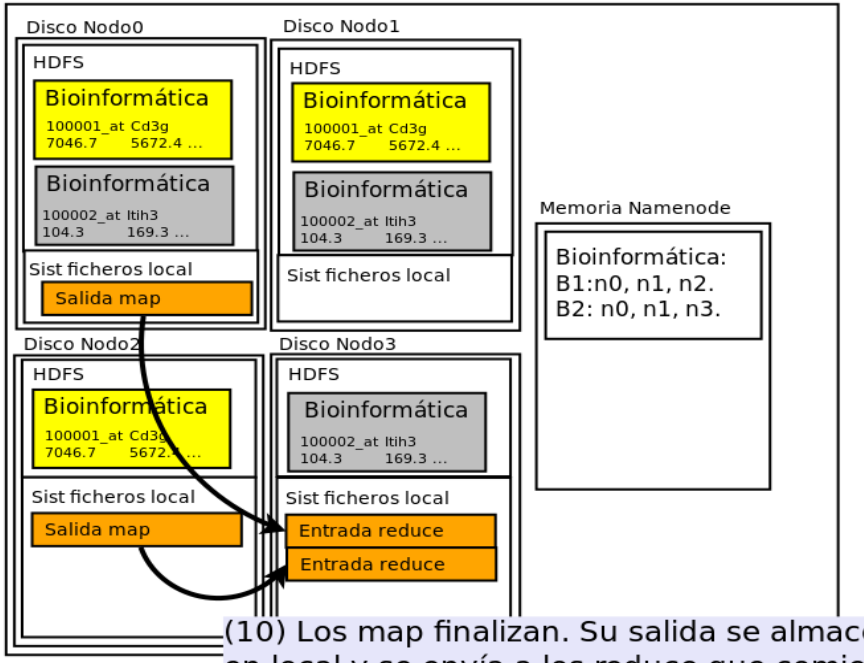
(9) Containers leen datos de entrada
y ejecutan los map





Select

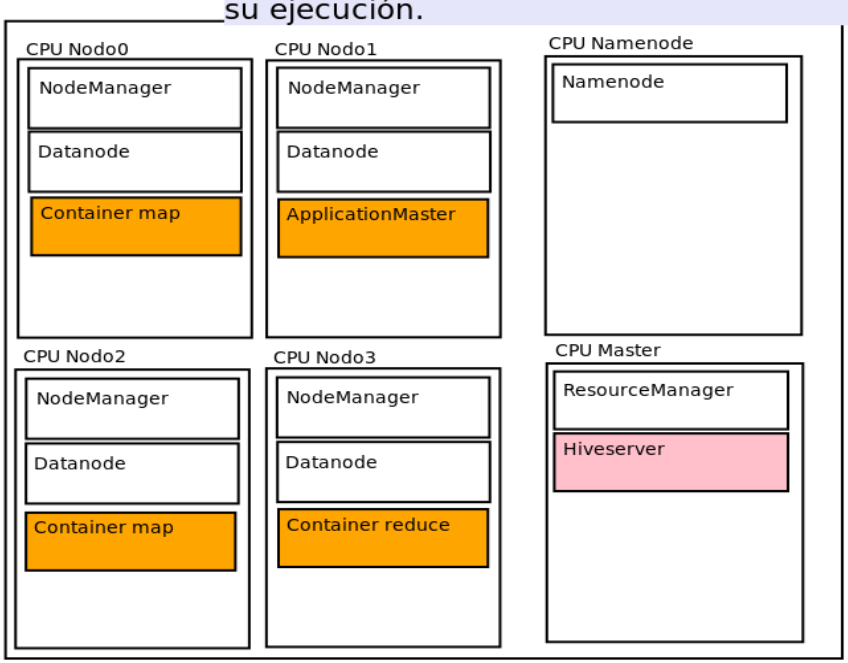
Almacenamiento
(HDFS)

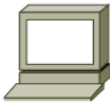


Cluster Hadoop

(10) Los map finalizan. Su salida se almacena en local y se envía a los reduce que comienzan su ejecución.

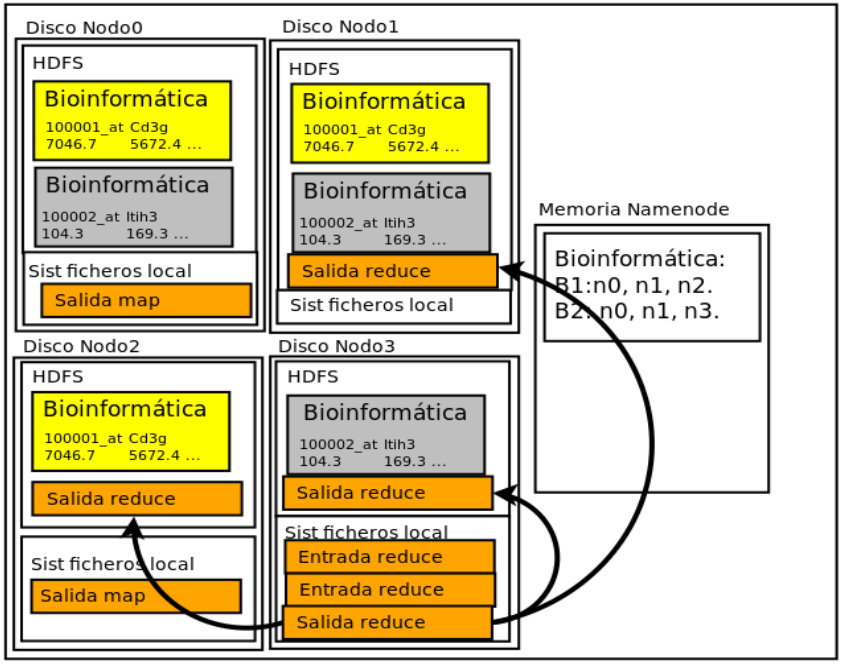
Procesamiento
(YARN Y MapReduce)





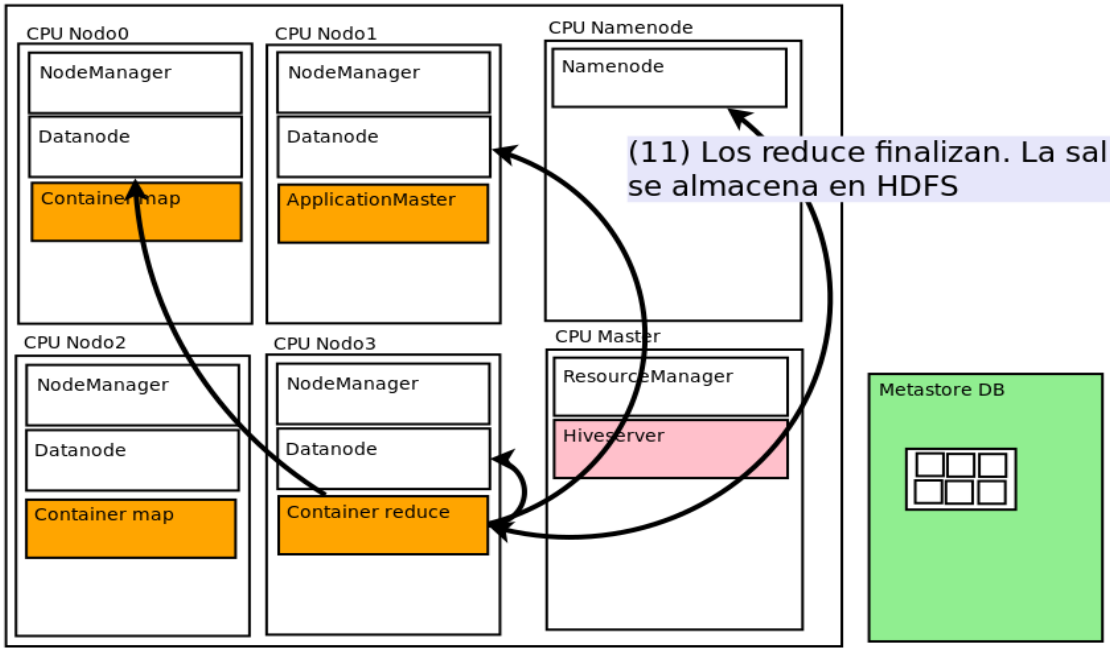
Select

Almacenamiento
(HDFS)



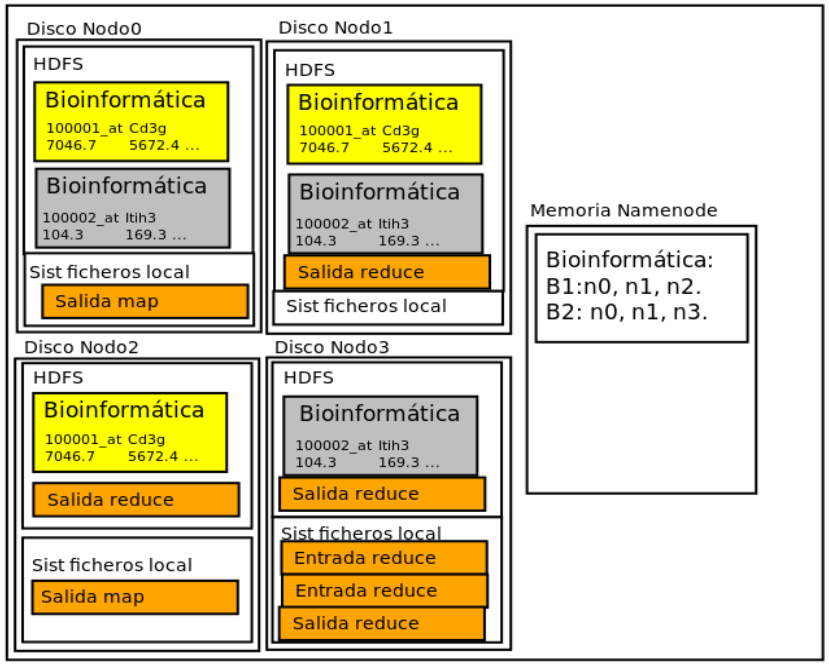
Cluster Hadoop

Procesamiento
(YARN Y MapReduce)



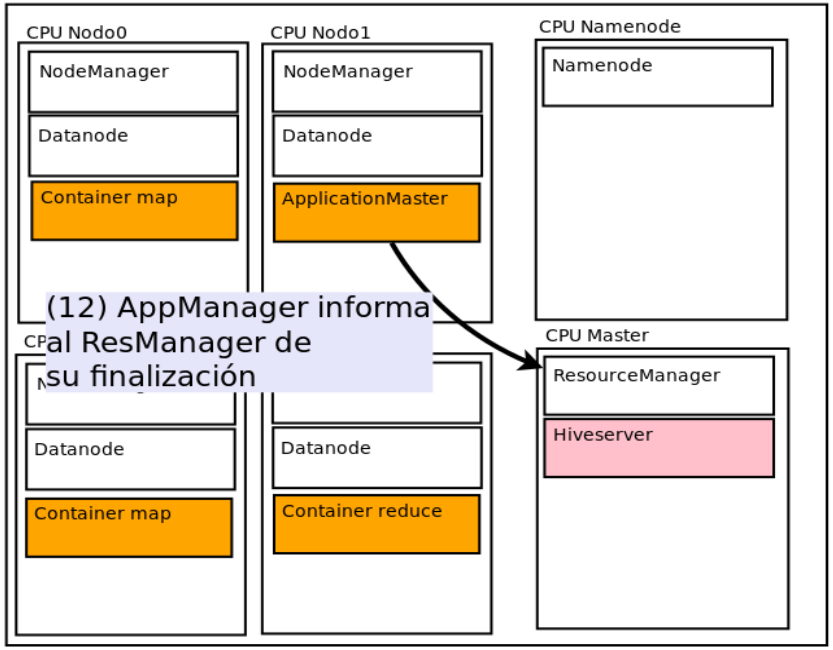
Select

Almacenamiento
(HDFS)



Cluster Hadoop

Procesamiento
(YARN Y MapReduce)

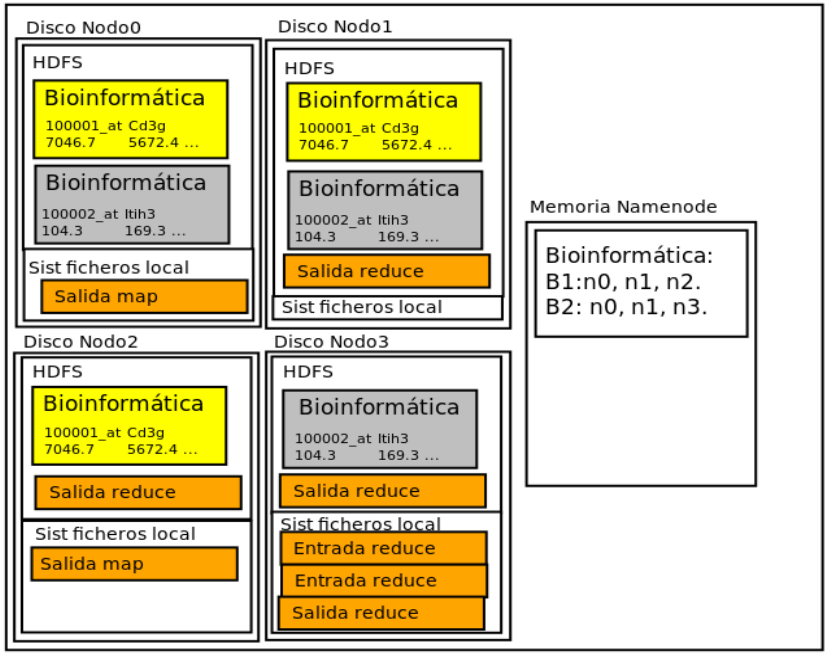




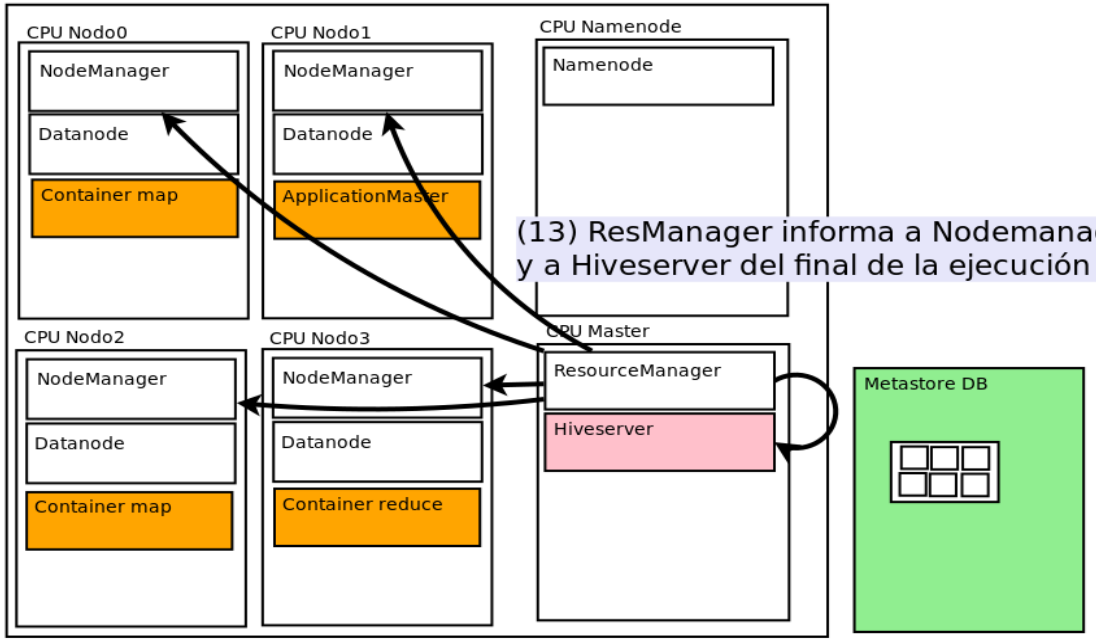
Select

Cluster Hadoop

Almacenamiento
(HDFS)



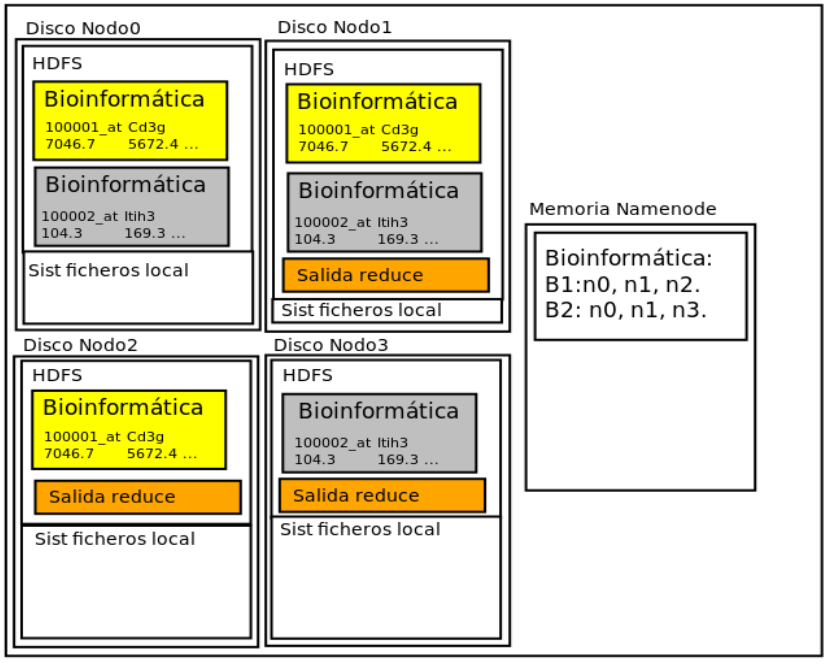
Procesamiento
(YARN Y MapReduce)





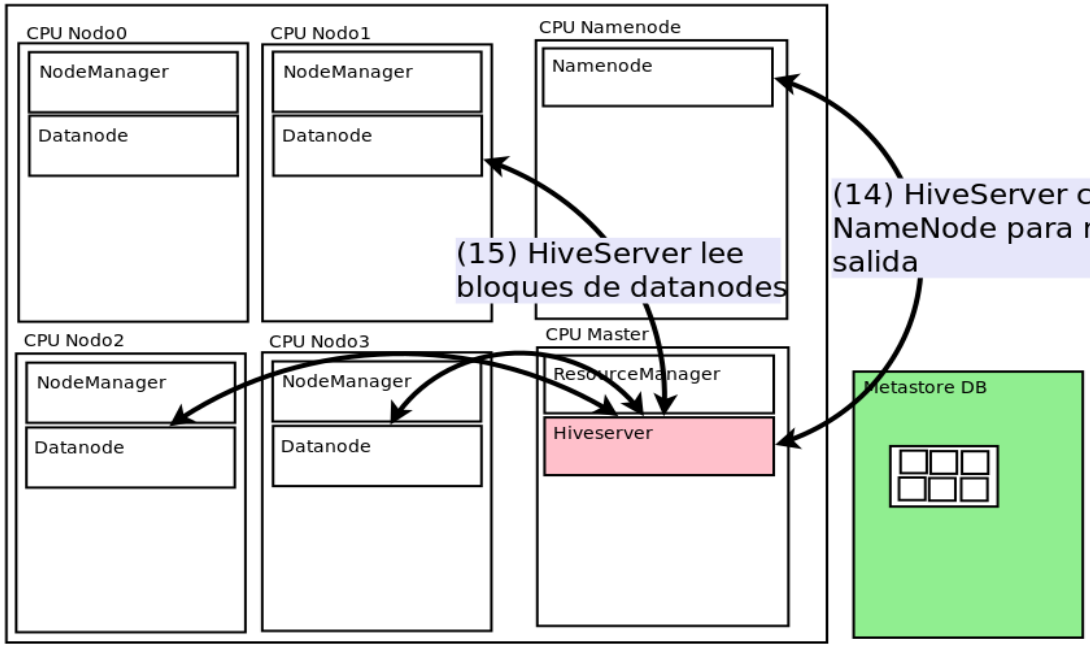
Select

Almacenamiento
(HDFS)



Cluster Hadoop

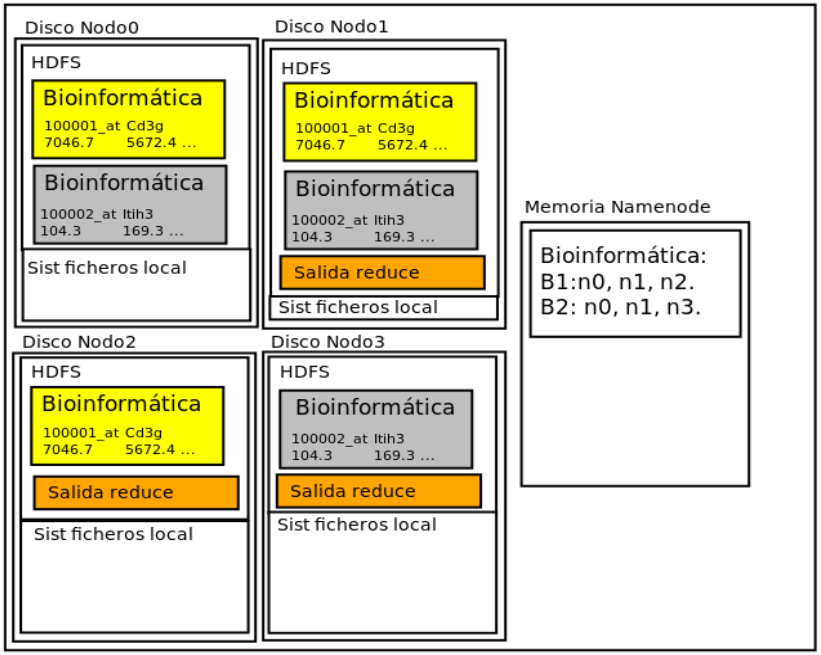
Procesamiento
(YARN Y MapReduce)





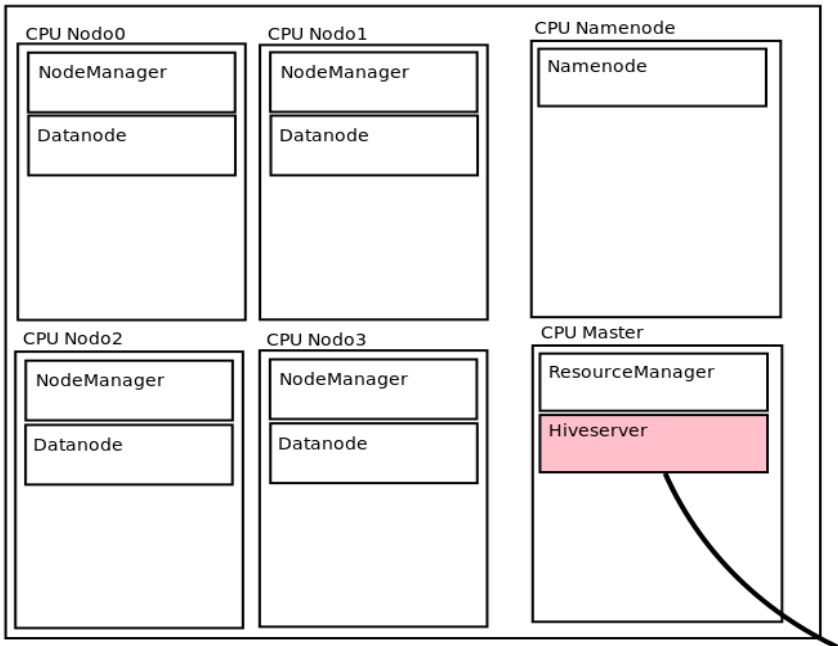
Select

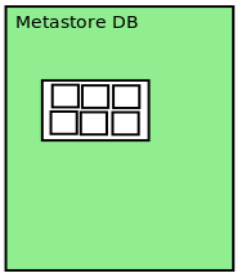
Almacenamiento
(HDFS)



Cluster Hadoop

Procesamiento
(YARN Y MapReduce)



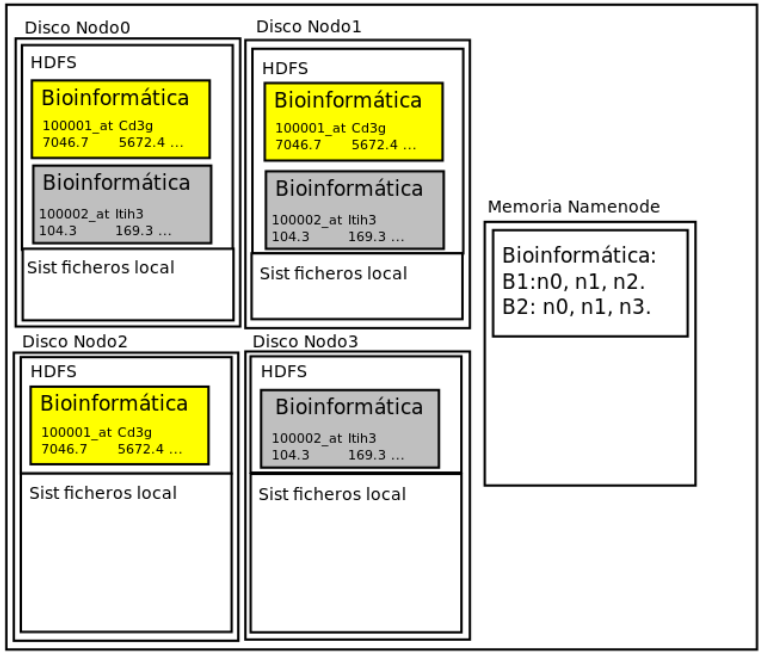




(16) HiveServer revuelve resultados al cliente

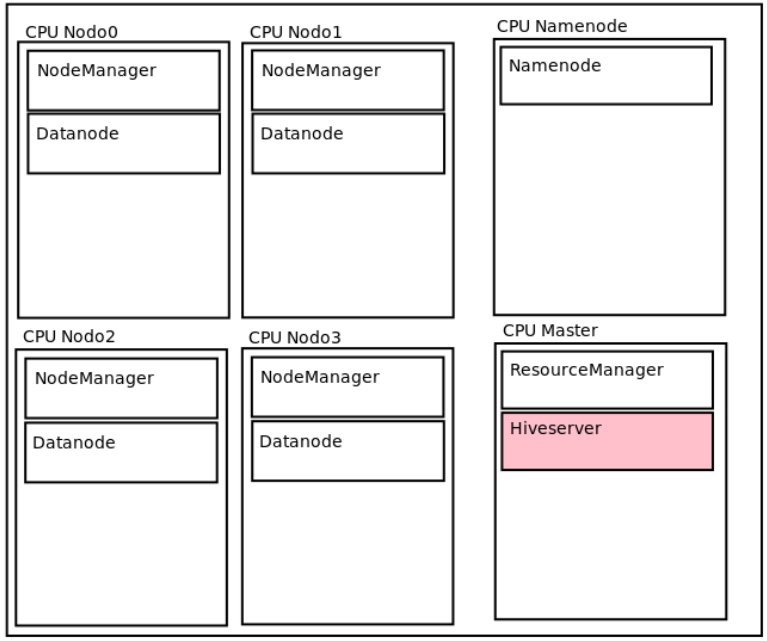
CLUSTER

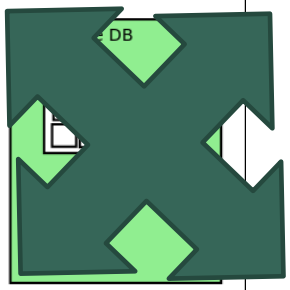
Almacenamiento (HDFS)



Cluster Hadoop

Procesamiento (YARN Y MapReduce)





¿Qué pasa si falla?

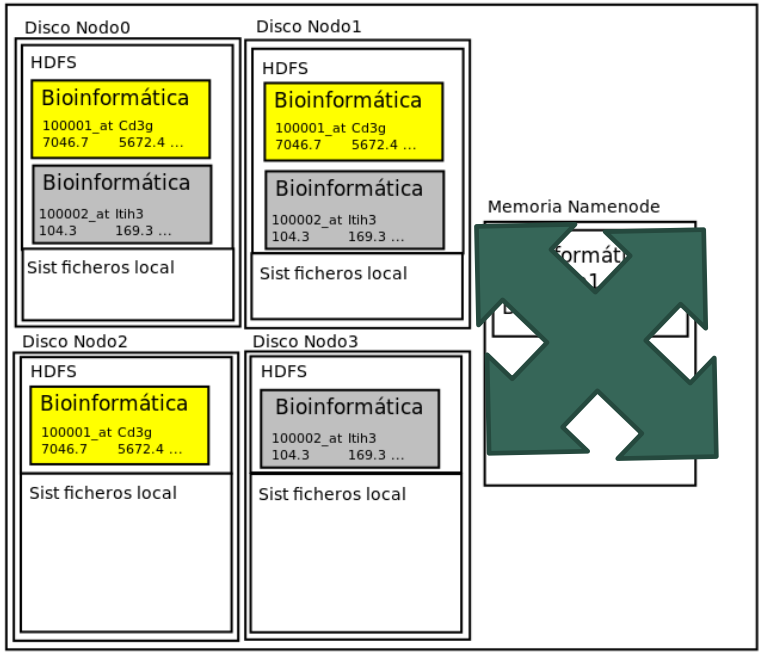
- MetastoreDB

CLUSTER

¿Qué pasa si falla?

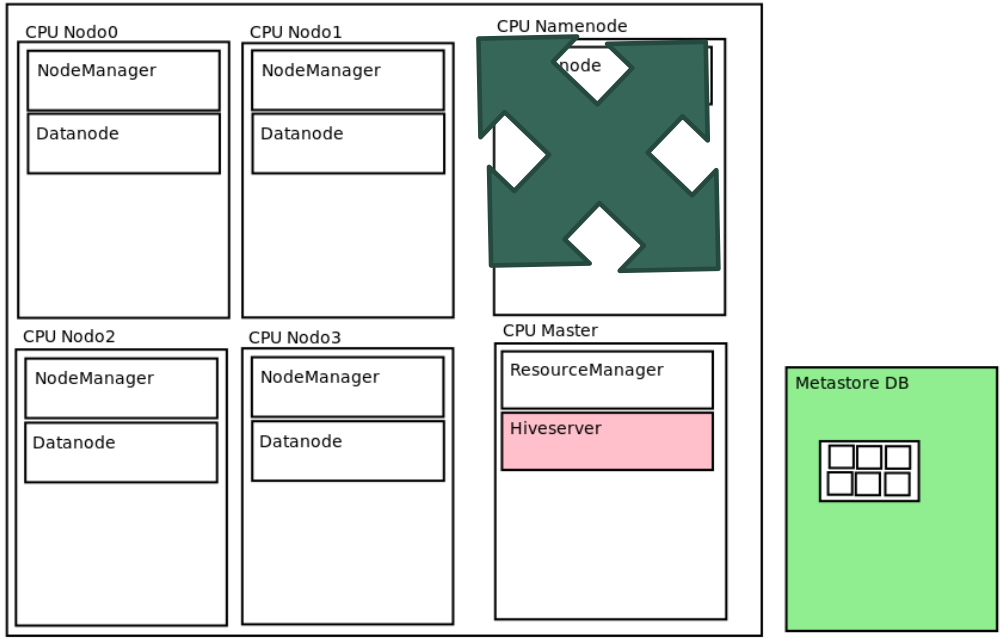
- NameNode

Almacenamiento (HDFS)



Cluster Hadoop

Procesamiento (YARN Y MapReduce)



CLUSTER

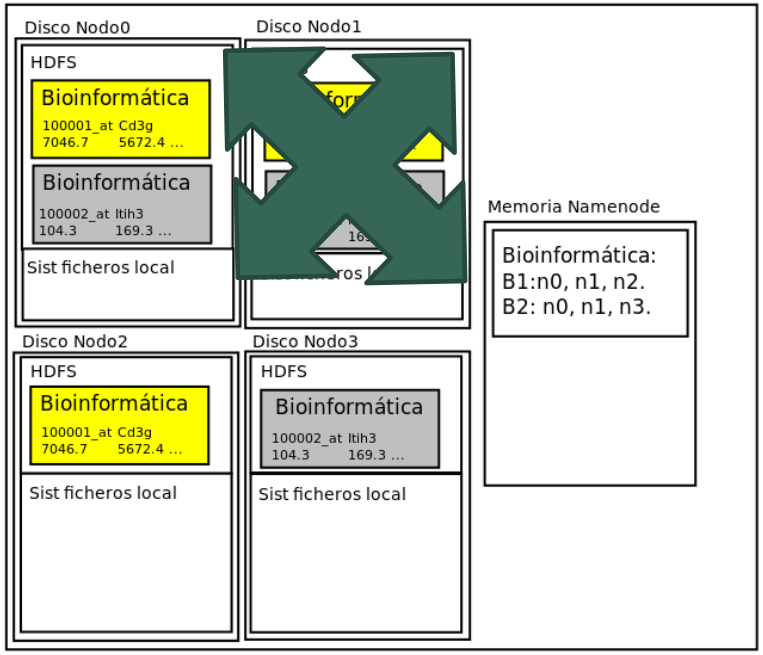
¿Qué pasa si falla?

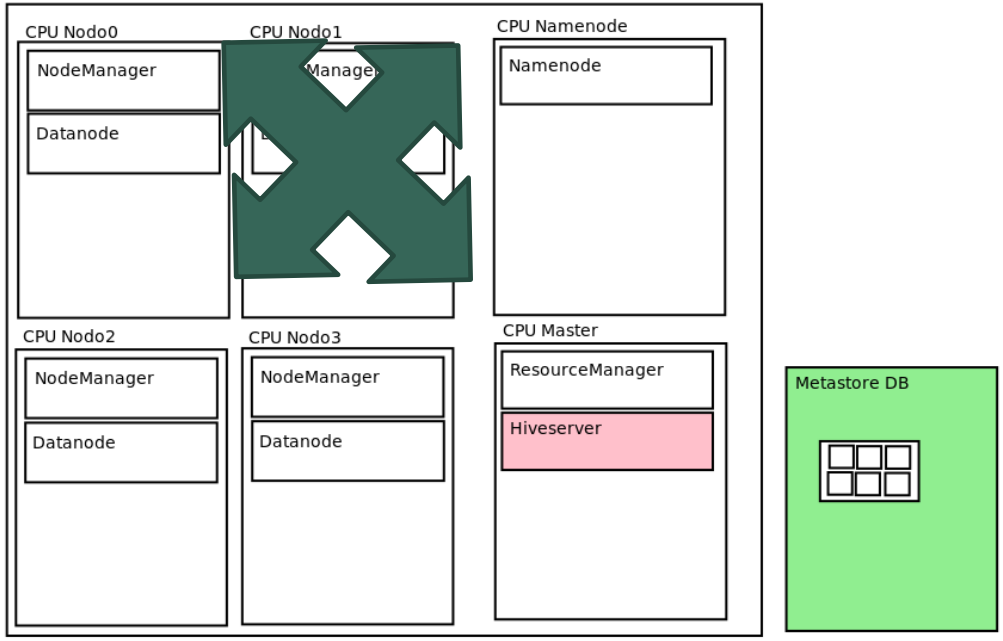
- Datanode I

Almacenamiento (HDFS)

Procesamiento (YARN Y MapReduce)

Cluster Hadoop

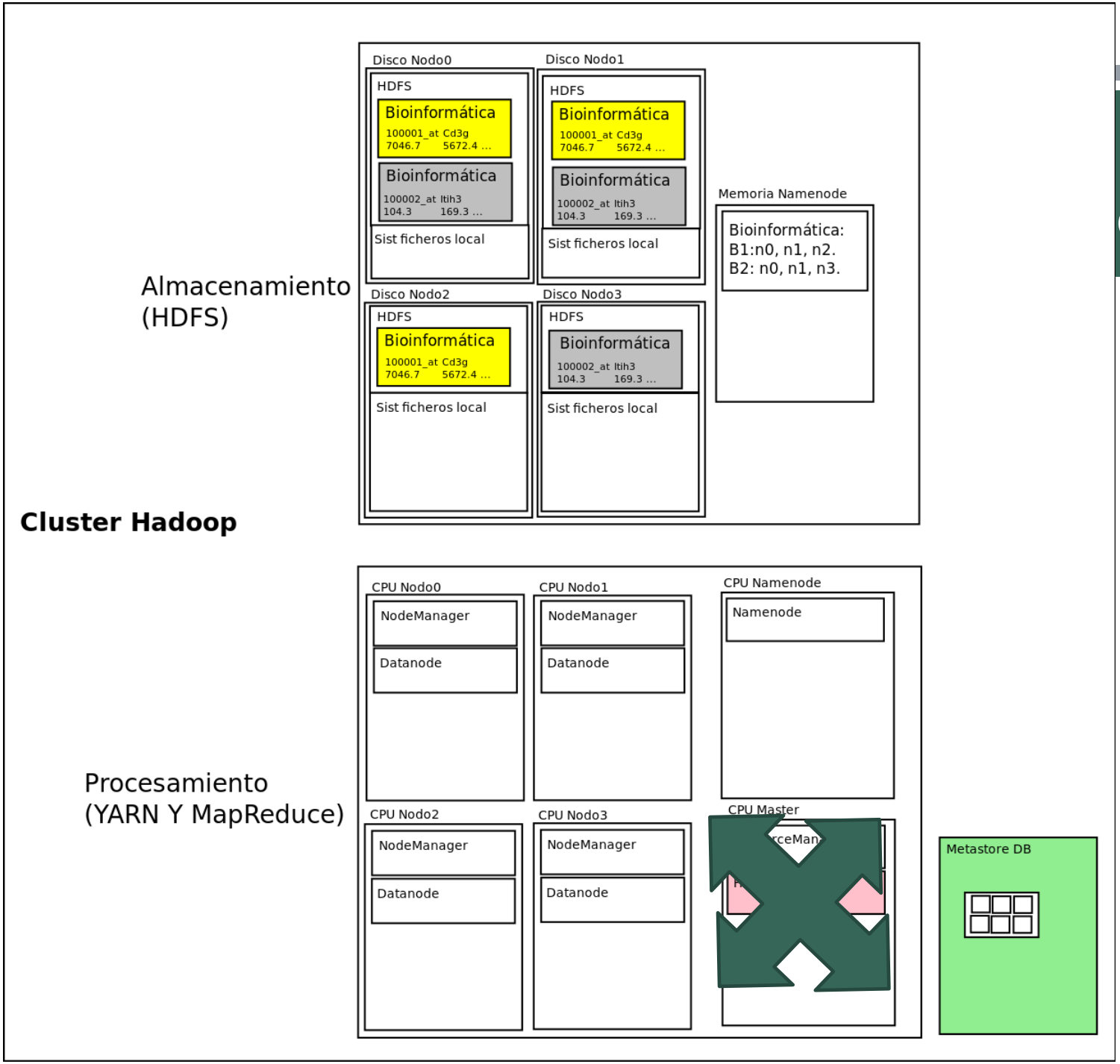




CLUSTER

¿Qué pasa si falla?

- Master



TRABAJANDO CON UNA BASE DE DATOS

- Vamos a ver las órdenes del lenguaje de definición de datos (Data definition Language) de HiveQL.
- Creación de base de datos:
  - Orden:

    CREATE (DATABASE|SCHEMA) [IF NOT EXISTS] database_name
    [COMMENT database_comment]
    [LOCATION hdfs_path]
    [WITH DBPROPERTIES (property_name=property_value, ...)];

  - Ejemplo:

    create database if not exists miDB
    Comment 'mi primera DB'
    Location '/user/cloudera/midb'
    With dbproperties ('Creada por' = 'User', 'Creada el' = '11-Nov-2017');

TRABAJANDO CON UNA BASE DE DATOS

- Eliminación de base de datos:
  - Orden:

    DROP (DATABASE|SCHEMA) [IF EXISTS] database_name [RESTRICT|CASCADE];

    - RESTRICT: no elimina la base de datos si contiene tablas (opción por defecto).
    - CASCADE: elimina las tablas que contiene la base de datos.
  - Ejemplo:

    drop database if exists miDB cascade;

TRABAJANDO CON UNA BASE DE DATOS

- Modificar las propiedades (‘dbproperties’) de una base de datos:
  - Orden:

    ALTER (DATABASE|SCHEMA) database_name SET DBPROPERTIES (property_name=property_value, ...);
  - Ejemplo:

    Alter database miDB set dbproperties ('Creada por' = 'User1', 'Creada el' = ‘1-Dic-2017');

TRABAJANDO CON UNA BASE DE DATOS

- Utilizar una base de datos:
  - Orden:

    USE (DATABASE|SCHEMA) database_name;
  - Ejemplo:

    use miDB;
- Mostrar bases de datos:
  - Orden:

    SHOW (DATABASES|SCHEMAS) [LIKE identifier_with_wildcards];
  - Ejemplo:

    Show databases like 'mi*'

TRABAJANDO CON UNA BASE DE DATOS

- Describir una base de datos:
  - Orden:

    DESCRIBE DATABASE [EXTENDED] db_name;

    DESCRIBE SCHEMA [EXTENDED] db_name;
  - Ejemplos:

    Describe database miDB;

    Describe database extended miDB;

TRABAJANDO CON UNA BASE DE DATOS

- Prueba los ejercicios de primeros pasos con Hive.
  - Creación de base de datos, modificación, utilización, borrado.
  - hive-1erpasos.ipynb

MODELO DE DATOS

- Hive soporta una variedad de tipos de datos que se dividen en dos categorías:
  - Primitivos.
    - Numéricos, string, fecha/hora, misceláneos.
  - Complejos.
    - STRUCT.
    - MAP.
    - ARRAY.

TIPOS DE DATOS PRIMITIVOS

- Tipos de datos numéricos:
  - TINYINT, SMALLINT, INT, BIGINT,FLOAT, DOUBLE, DECIMAL.
- Tipos de datos string:
  - STRING, CHAR,VARCHAR.
- Tipos de datos de fecha/hora:
  - DATE, TIMESTAMP.
- Tipos de datos misceláneos:
  - BOOLEAN, BINARY.

TIPOS DE DATOS COMPLEJOS

- STRUCT:
  - Similar al tipo de dato struct de C. Es un tipo de registro que almacena un conjunto de valores nombrados, cada uno de los cuales puede ser de cualquier tipo de datos primitivo.
  - Los campos en un STRUCT se acceden utilizando la notación del punto "."
  - Sintaxis:
    - STRUCT<col_name : data_type [COMMENT col_comment], ...>
    - Por ejemplo, si una columna llamada "dirección" es de tipo STRUCT {ciudad STRING; pais STRING}, entonces podemos referenciar al campo ciudad como direccion.ciudad.

TIPOS DE DATOS COMPLEJOS

- MAP:
  - Contiene pares clave.valor, en los que los elementos se acceden utilizando su clave.
  - Por ejemplo, si una columna "nombrecompleto" es de tipo MAP, 'nombre' -> 'pepe' y 'apellido' -> 'perez', entonces el apellido se puede acceder utilizando nombrecompleto['apellido'].
- ARRAY:
  - Secuencia ordenada de elementos similares. Mantiene un índice para acceder a los elementos.
  - Por ejemplo, un ARRAY llamado "días" que contiene los elementos ['Lunes', 'Martes', 'Miércoles', 'Jueves', 'Viernes', 'Sábado', 'Domingo'], el primer día (Lunes) se puede acceder mediante dias[0].
- UNIONTYPE:
  - Es similar al tipo Unions en C. Permite almacenar diferentes tipos de datos en la misma localización de memoria.
  - Se puede definir un UNIONTYPE con varios tipos, pero en cada momento solamente podrá haber un único tipo de dato almacenado.

OPERADORES

- Hive proporciona los siguientes tipos de operadores:
  - Relacionales.
    - Ejemplos: A = B, A != B, A > B, …
  - Aritméticos.
    - Ejemplos: A + B, A * B, …
  - Lógicos.
    - Ejemplos: A AND B, A OR B, NOT A, …
  - Complejos.
    - Ejemplos:
      - A[i]: devuelve el elemento «i» del array «A».
      - M[key]: «M» es un objeto de tipo map.
      - S.a: «S» es un objeto de tipo struct con un campo llamado «a».

TABLAS

- En Hive, las tablas son entidades «virtuales», ya que los datos se encuentran almacenados en ficheros en HDFS.
- Existen dos tipos de tablas en Hive:
  - Tablas gestionadas (*managed tables*): también llamadas **tablas internas**:
    - Hive controla la tabla por completo. Sus datos están almacenados en un subdirectorio gestionado únicamente por Hive.
    - Cuando eliminamos la tabla, se eliminan también sus datos.
  - **Tablas externas**:
    - Los datos están almacenados fuera del control de Hive.
    - Cuando se elimina la tabla, solamente se eliminan los metadatos, mientras que los datos persisten.

TABLAS

- **¿Qué tipo de tabla (interna o externa) sería necesaria en el siguiente caso?**
  - Deseamos realizar una captura de datos en tiempo real, y poder realizar consultas sobre ellos utilizando Hive **al mismo tiempo que los datos se están generando.**

TABLAS EN EL DDL

- Crear tablas:
  - Orden:

```
CREATE [TEMPORARY] [EXTERNAL] TABLE [IF NOT EXISTS]
  [db_name.] table_name
  [(col_name data_type [COMMENT col_comment], ...)]
  [COMMENT table_comment]
  [PARTITIONED BY (col_name data_type [COMMENT col_comment], ...)]
  [CLUSTERED BY (col_name, col_name, ...) [SORTED BY (col_name [ASC|DESC], ...)] INTO num_buckets BUCKETS]
  [SKEWED BY (col_name, col_name, ...)
  ON ((col_value, col_value, ...), (col_value, col_value, ...), ...)
  [STORED AS DIRECTORIES]
  [    [ROW FORMAT row_format]
  [STORED AS file_format]
  | STORED BY 'storage.handler.class.name' [WITH SERDEPROPERTIES (...)]    ]
  [LOCATION hdfs_path]
  [TBLPROPERTIES (property_name=property_value, ...)]
  [AS select_statement];
```

TABLAS EN EL DDL

- Parámetros:
  - [TEMPORARY]: Sirve para crear una tabla temporal que desaparecerá cuando finalice la sesión actual.
  - [EXTERNAL]: Sirve para crear tablas externas.
  - [IF NOT EXISTS]: Para crear la tabla solamente si no existe anteriormente.
  - [db_name]: Sirve para indicar a qué base de datos pertenece la tabla.
  - [COMMENT col_comment]: Para incluir comentarios en una columna. El comentario debe ir entre comillas simples.
  - [COMMENT table_comment]: Se utiliza para añadir comentarios a la tabla. El comentario debe ir entre comillas simples.
  - [PARTITIONED BY]: Sirve para crear tablas particionadas. Puede haber más de una columna de partición en una tabla..
  - [CLUSTERED BY]: Para crear buckets. Se pueden crear buckets en la tabla utilizando las columnas especificadas aquí.
  - [SKEWED BY]: Mejora las prestaciones de las tablas que tienen valores sesgados. Cuando se utiliza esta opción, los valores que aparecen con frecuencia se reparten entre varios ficheros, mientras que el resto de valores van a otro fichero.
  - [LOCATION hdfs_path]: Se utiliza cuando creamos tablas externas. Es la localización de los datos que referencia la tabla externa.
  - [TBLPROPERTIES]: Permite insertar información sobre la tabla en forma de pares <clave,valor>.
  - [AS select_statement]: Create Table As Select, conocido como CTAS, se utiliza para crear una tabla basada en la salida de una consulta.

TABLAS EN EL DDL

- Eliminar un tabla:
  - Orden:

    DROP TABLE [IF EXISTS] table_name [PURGE];
  - Ejemplo:

    Drop table if exists mitabla;
- Vaciar una tabla:
  - Orden:

    TRUNCATE TABLE table_name [PARTITION partition_spec];
  - Ejemplo:

    Truncate table mitabla;

TABLAS EN EL DDL

- Renombrar tabla:
  - Orden:

    ALTER TABLE table_name RENAME TO new_table_name;
  - Ejemplo:

    ALTER TABLE mitabla RENAME TO mitabla2;
- Modificar las propiedades de una tabla:
  - Orden:

    ALTER TABLE table_name SET TBLPROPERTIES table_properties;
  - Ejemplo:

    ALTER TABLE mitabla SET TBLPROPERTIES (‘Comentario' = ‘Propiedades de la tabla actualizadas');

TABLAS EN EL DDL

- Mostrar las propiedades de una tabla:
  - Orden:

    SHOW TBLPROPERTIES table_name;
  - Ejemplo:

    SHOW TBLPROPERTIES mitabla;
- Mostrar la orden de creación de una tabla:
  - Orden:

    SHOW CREATE TABLE ([db_name.]table_name|view_name);
  - Ejemplo:

    SHOW CREATE TABLE mitabla;

TABLAS EN EL DDL

- Mostrar las tablas de una bases de datos:
  - Orden:

SHOW TABLES [IN database_name] ['identifier_with_wildcards'];

  - Ejemplo:

Show tables:

- Mostrar las particiones de una tabla:
  - Orden:

SHOW PARTITIONS [db_name.]table_name [PARTITION(partition_spec)];

  - Ejemplo:

Show partitions mitabla;

TABLAS EN EL DDL

- Prueba los ejercicios de creación de tablas con campos de tipos básicos y tipos complejos.
  - Hive-tablastiposbasicos-estudiantes.ipynb
  - hive-tablastiposcompuestos-estudiantes.ipynb

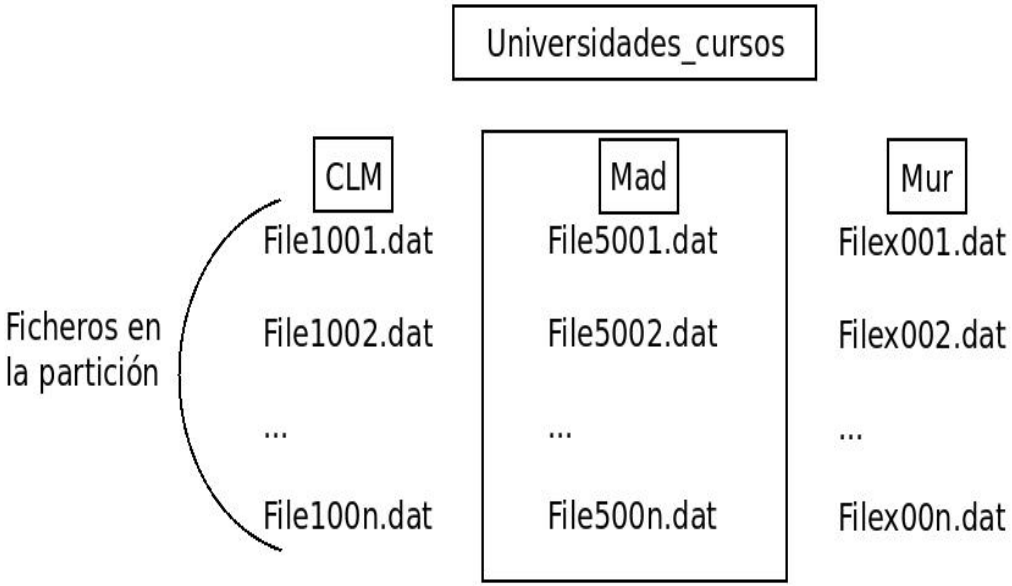
PARTICIONES

- Definen la forma en que Hive almacena físicamente los datos en disco.
- Permiten agrupar las filas basándose en el valor de alguna columna de interés.
- Es de interés para mejorar la eficiencia de las consultas que requieren escaneos completos de tablas.
- **Una partición en Hive es una carpeta en HDFS.**

PARTICIONES

```
CREATE TABLE Universidades_cursos
.. .
PARTITIONED BY (Region String)
```

```
SELECT nombreCurso, nombreUni, ects
FROM Universidades_cursos
WHERE Region = 'Mad'
```



PARTICIONES

- Prueba los ejercicios de creación de tablas con particiones.
  - hive-tablasparticiones.ipynb

BUCKETS

- Buckets:
  - Complementan a las particiones para organizar mejor los datos y de esta forma realizar de forma más eficiente las consultas.
  - Al contrario que las particiones, se define un número concreto de buckets, que no se puede modificar tras crear la tabla.
  - Los buckets están basados en que items con el mismo valor en una función hash van al mismo bucket.

BUCKETS

Particiones	Buckets
Carpeta en HDFS	Fichero en HDFS
Número indefinido de particiones	Número definido de buckets
La columna sobre la que realizamos la partición no se incluye en el esquema de la tabla de Hive	La columna sobre la que realizamos el bucket sí se incluye en el esquema de la tabla de Hive
Número de particiones variable	Una vez creada la tabla, no se pueden modificar sus buckets
Tamaños de particiones indiferentes	Idealmente, todos los buckets tienen el mismo tamaño.

BUCKETS

- Prueba los ejercicios de creación de tablas con buckets.
  - hive-tablasbuckets.ipynb

VISTAS

- Las vistas son tablas virtuales que constituyen una ventana hacia la tabla subyacente (conocida como tabla «base»).
- Al acceder a una vista, estamos accediendo a la tabla base.
- Crear vistas:
  - Orden:

    CREATE VIEW [IF NOT EXISTS] view_name [(column_name [COMMENT column_comment], ...)]

    [COMMENT view_comment]

    [TBLPROPERTIES (property_name = property_value, ...)]

    AS SELECT ...;
  - Ejemplo:

    Create view if not exists mivista

    As select id, firstname from mitabla Where firstname = 'John';

VISTAS

- Eliminar vistas:
  - Orden:

    DROP VIEW [IF EXISTS] view_name;
  - Ejemplo:

    Drop view mivista;
- Modificar propiedades de vistas:
  - Orden:

    ALTER VIEW view_name SET TBLPROPERTIES table_properties;
  - Ejemplo:

    Alter View mivista SET TBLPROPERTIES ('Comentario' = 'Propiedades de la vista actualizadas');
- Cambiar una vista:
  - Orden:

    ALTER VIEW view_name AS select_statement;
  - Ejemplo:

    ALTER VIEW mi vista AS select id, firstname from mitabla;

VISTAS

- Prueba los ejercicios de creación de vistas.
  - hive-creandovistas.ipynb

LENGUAJE DE MANIPULACIÓN DE DATOS (DML)

- Cargar ficheros en tablas:
  - Orden:

    LOAD DATA [LOCAL] INPATH 'filepath' [OVERWRITE] INTO TABLE tablename [PARTITION (partcol1=val1, partcol2=val2 ...)]
  - Parámetros:
    - [LOCAL]: Si se especifica, busca el fichero en el sistema de archivos local (no en HDFS).
    - FILEPATH: La ruta del archivo de interés.
    - [OVERWRITE]: Si es especifica, se eliminan los datos de la tabla antes de insertar estos.
    - tablename: El nombre de la tabla.
    - [PARTITION (partcol1=val1, partcol2=val2 ...)]: Para indicar particiones.

LENGUAJE DE MANIPULACIÓN DE DATOS (DML)

- Insertar datos en tablas:
  - Orden:

INSERT OVERWRITE TABLE tablename [PARTITION (partcol1=val1, partcol2=val2 ...) [IF NOT EXISTS]] select select_statement FROM from_statement;

INSERT INTO TABLE tablename [PARTITION (partcol1=val1, partcol2=val2 ...)] select select_statement FROM from_statement;

INSERT INTO TABLE table_name [PARTITION (partcol1[=val1], partcol2[=val2] ...)] VALUES values_row [, values_row ...]

  - Ejemplo:

INSERT OVERWRITE TABLE ventas SELECT * FROM comercios ;

INSERT INTO ventas SELECT * FROM comercios WHERE cuidad= 'Madrid';

INSERT INTO ventas VALUES (1, 'Jose', 'Garcia', 'H-43 Sector-23', 'Delhi', 'India', '10.10.10.10', 'P_1', '15-11-1985');

LENGUAJE DE MANIPULACIÓN DE DATOS (DML)

- Realizar consultas de selección
  - Orden:

```
SELECT [ALL | DISTINCT] select_expr, select_expr, ...
  FROM table_reference
  [WHERE where_condition]
  [GROUP BY col_list]
  [ORDER BY col_list]
  [CLUSTER BY col_list
    | [DISTRIBUTE BY col_list] [SORT BY col_list]
  ]
 [LIMIT [offset,] rows]
```

Ejemplo:

```
Select * from customers where country='ES';
```

- Para dejar el resultado en HDFS:
  - INSERT OVERWRITE DIRECTORY /ruta/al/directorio SELECT count(*) FROM customers WHERE country='ES';

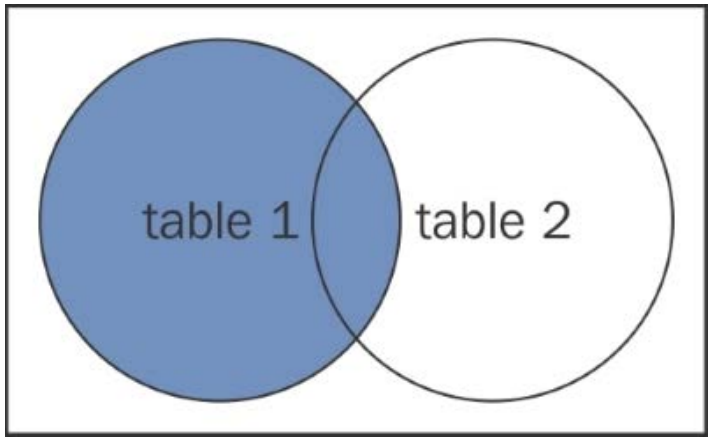
REUNIONES

- Una reunión (join) se utiliza para recuperar datos de interés de dos o más tablas basados en un campo común.
- Hive solamente soporta reuniones de igualdad.
- Poner a la derecha la tabla más grande (si es posible) para realizar un mapjoin si la tabla pequeña cabe en memoria de los mappers.
- Sintaxis:

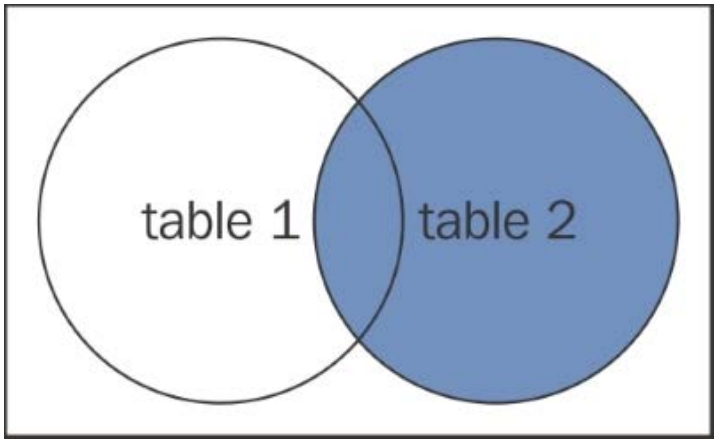
```
join_table:
    table_reference JOIN table_factor [join_condition]
  | table_reference {LEFT|RIGHT|FULL} [OUTER] JOIN table_reference    join_condition
  | table_reference LEFT SEMI JOIN table_reference join_condition
  | table_reference CROSS JOIN table_reference [join_condition]
```

  - Donde…
    - table_reference: El nombre de la tabla que vamos a reunir.
    - table_factor: Puede ser tanto un nombre de tabla como una subconsulta.
    - join_condition: Es la condición en función de la cual se va a producir la reunión.

REUNIONES

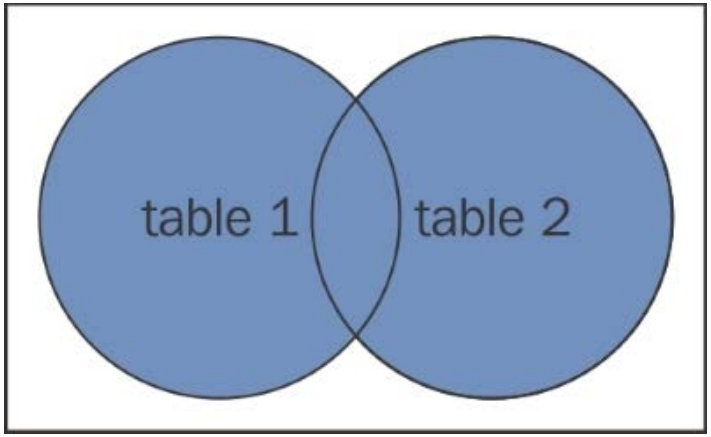


Left join



Right join

Full join



REUNIONES

- Ejemplos:

```
SELECT a.* FROM tabla1 JOIN tabla2 b ON a.id = b.id;

SELECT a.*, b.* FROM tabla1 a JOIN tabla2 b ON a.id = b.id;

SELECT a.* FROM tabla1 a JOIN tabla2 b ON a.id = b.id and a.fname =
b.fname;

SELECT * FROM tabla1 a LEFT OUTER JOIN tabla2 b ON a.id = b.id;

SELECT * FROM tabla1 a RIGHT OUTER JOIN tabla2 b ON a.id = b.id;

SELECT * FROM tabla1 a FULL OUTER JOIN tabla2 b ON a.id = b.id;
```

REUNIONES

- Prueba los ejercicios sobre reuniones.
  - hive-reuniones.ipynb

FORMATOS DE SERIALIZACIÓN Y DESERIALIZACIÓN (SERDE)

- Serialización y deserialización (o serde) consiste leer y escribir datos de una variedad de formatos.
- Algunos de los serdes que proporciona Hive por defecto son:
  - LazySimpleSerDe: Básico, se usa por defecto.
  - RegexSerDe: Utiliza expresiones regulares.
  - AvroSerDe: Utiliza el formato de archivo Avro.
  - OrcSerde: Utiliza el formato de archivo ORC.
  - ParquetHiveSerDe: Utiliza el formato de archivo Parquet.
  - JSONSerDe: Utiliza el formato de archivo JSON.
  - CSVSerDe: Utiliza el formato de archivo CSV.

FORMATOS DE SERIALIZACIÓN Y DESERIALIZACIÓN (SERDE)

- Prueba los ejercicios sobre Serde.
  - hive-serdes.ipynb

PLANES DE EJECUCIÓN DE CONSULTAS

- Para recuperar el plan de ejecución de una consulta pero sin ejecutarla.
- Orden:
  - EXPLAIN orden;
- Ejemplo:
  - EXPLAIN SELECT * FROM dataclean;

- Salida de ejemplo:

```
Explain
STAGE DEPENDENCIES:
  Stage-0 is a root stage
STAGE PLANS:
  Stage: Stage-0
    Fetch Operator
      limit: -1
      Processor Tree:
        TableScan
          alias: datacleanexternal
          Statistics: Num rows: 10321 Data size: 1986152 Basic stats: COMPLETE Column stats: NONE
          Select Operator
            expressions: idref (type: string), ident (type: string), gsm19023 (type: float), gsd19024 (type: float), gsd19025 (type: float), gsd19026 (type: float), genetitle (type: string), genesymbol (type: string), geneid (type: smallint), unigenetitle (type: string), unigenesymbol (type: string), unigeneid (type: string), nucleotidetitle (type: string)
            outputColumnNames: _col0, _col1, _col2, _col3, _col4, _col5, _col6, _col7, _col8, _col9, _col10, _col11, _col12
            Statistics: Num rows: 10321 Data size: 1986152 Basic stats: COMPLETE Column stats: NONE
            ListSink
```

PLANES DE EJECUCIÓN DE CONSULTAS

- Calcular estadísticas de tablas, para el cálculo de los planes de ejecución:
- Orden:
  - ANALYZE TABLE tabla COMPUTE STATISTICS;
- Ejemplo:
  - ANALYZE TABLE datacleanexternal COMPUTE STATISTICS;
- Salida de ejemplo:
  - …
  - INFO : Table default.datacleanexternal stats: [numFiles=12, numRows=10321, totalSize=1996473, rawDataSize=1986152]
  - …

TRANSACCIONES

- Hive proporciona ACID a nivel de fila, de forma que una aplicación puede añadir filas mientras que otra lee de la misma partición sin interferir.
- Limitaciones:
  - Transacciones dehabilitadas por defecto.
  - BEGIN, COMMIT, y ROLLBACK no están soportadas aún, las operaciones del lenguaje son auto-commit.
  - Solamente sobre tablas con buckets y en formato ORC.
  - …
- https://cwiki.apache.org/confluence/display/Hive/Hive+Transactions
- Prueba los ejercicios sobre transacciones.
  - hive-transacciones.ipynb

OTROS ASPECTOS INTERESANTES DE HIVE

- Extensibilidad:
  - Hive ofrece la posibilidad de implementar nuevas funcionalidades, mediante funciones definidas por el usuario (User Defined Functions, UDFs) en Java, Python.
- Compresión:
  - Hive permite comprimir datos tanto finales como intermedios.



Hive

Agustín C. Caminero Herráez
Depto. de Sistemas de Comunicación y Control.
UNED
accaminero@scc.uned.es / @accaminero